M02 — Teoría completa: diseño REST e implementación con ASP.NET Core 8

Autor: JAIME GALLO

Punto 2.1 — Diseño de APIs REST

Módulo 2: Desarrollo de APIs REST con ASP.NET Core Objetivos de aprendizaje:

- Comprender los principios del estilo arquitectónico REST.
- Identificar los recursos como unidad central del diseño de una API REST.
- Comprender la relación entre recursos, URIs y representaciones.
- Comprender el principio de interfaz uniforme y su aplicación en ASP.NET Core.
- Añadir al proyecto AceriaApp un documento de diseño de APIs REST y una clase de validación de URIs REST.

Bloque 1 — Definición de REST

REST es un estilo arquitectónico definido por Roy Fielding en su tesis doctoral de 2000. No es un protocolo ni un estándar, sino un conjunto de restricciones que, aplicadas de forma conjunta, producen un sistema con propiedades específicas. Las restricciones son seis: cliente-servidor, sin estado, cacheable, interfaz uniforme, sistema en capas y código bajo demanda. Cada restricción aporta una propiedad al sistema. La restricción de cliente-servidor separa las responsabilidades de la interfaz de usuario y del almacenamiento de datos. La restricción de sin estado garantiza que cada petición contenga toda la información necesaria para ser procesada. La restricción de cacheable permite que las respuestas se almacenen para reducir la carga. La restricción de interfaz uniforme simplifica la arquitectura global.



Figura M02.2.1.1 — Las seis restricciones de REST y su función.

Restricciones de REST:

1. Cliente-servidor
2. Sin estado
3. Cacheable
4. Interfaz uniforme

5. Sistema en capas
6. Código bajo demanda (opcional)

- **Línea 1:** 1. Cliente-servidor → primera restricción, separa interfaz y almacenamiento.
- **Línea 2:** 2. Sin estado → segunda restricción, cada petición contiene toda la información.
- **Línea 3:** 3. Cacheable → tercera restricción, las respuestas se pueden almacenar.
- **Línea 4:** 4. Interfaz uniforme → cuarta restricción, simplifica la arquitectura.
- **Línea 5:** 5. Sistema en capas → quinta restricción, permite intermediarios.
- **Línea 6:** 6. Código bajo demanda → sexta restricción, opcional.

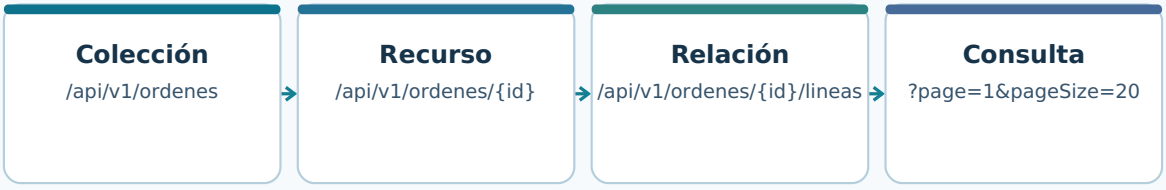
Las seis restricciones no son obligatorias por separado: son un conjunto. Un sistema que aplica cinco de las seis no es REST, es un sistema parcialmente REST. La restricción de interfaz uniforme es la que más impacto tiene en el diseño de una API. Se descompone en cuatro sub-restricciones: identificación de recursos, manipulación de recursos a través de representaciones, mensajes auto-descriptivos e hipermedia como motor del estado de la aplicación. Las cuatro sub-restricciones guían el diseño de las URIs y de los métodos HTTP. El curso aplica las cuatro sub-restricciones al diseño de la API de la acería.

La aplicación de las seis restricciones produce un sistema con propiedades medibles: escalabilidad, simplicidad, modificabilidad y visibilidad. La propiedad de escalabilidad se obtiene de la restricción sin estado y de la restricción cacheable. La propiedad de simplicidad se obtiene de la restricción de interfaz uniforme. La propiedad de modificabilidad se obtiene de la restricción de sistema en capas. La propiedad de visibilidad se obtiene de la restricción de mensajes auto-descriptivos. Las cuatro propiedades son las razones por las que REST se ha convertido en el estilo dominante para APIs web.

Bloque 2 — Recursos como unidad central del diseño

Un recurso es cualquier entidad identificable que puede ser referenciada y manipulada. En una API REST, los recursos son la unidad central del diseño. Una orden es un recurso. Un material es un recurso. Un cliente es un recurso. La colección de órdenes también es un recurso. Cada recurso se identifica con una URI única. La URI no describe acciones, describe entidades. La diferencia entre una URI que describe acciones y una URI que describe entidades es la diferencia entre RPC y REST. Una URI como /api/crearOrden describe una acción. Una URI como /api/v1/ordenes describe una entidad.

Del recurso a la URI



La flecha indica una relación de diseño o una secuencia de trabajo.

Figura M02.2.1.2 — De la colección al recurso y sus relaciones.

URIs de recursos:

- **/api/v1/ordenes** → colección de órdenes.
- **/api/v1/ordenes/{id}** → orden concreta.
- **/api/v1/ordenes/{id}/lineas** → líneas de una orden concreta.

- **/api/v1/materiales** → colección de materiales.
- **/api/v1/clientes** → colección de clientes.
- **Línea 1:** /api/v1/ordenes → recurso de colección de órdenes.
- **Línea 2:** /api/v1/ordenes/{id} → recurso de orden concreta.
- **Línea 3:** /api/v1/ordenes/{id}/lineas → recurso de líneas de una orden.
- **Línea 4:** /api/v1/materiales → recurso de colección de materiales.
- **Línea 5:** /api/v1/clientes → recurso de colección de clientes.

La identificación de recursos con URIs tiene tres consecuencias prácticas. La primera es que las URIs se construyen con sustantivos en plural para las colecciones. La segunda es que las relaciones entre recursos se expresan con anidamiento de URIs. La tercera es que las URIs no contienen verbos. Las tres consecuencias se aplican de forma uniforme en toda la API. La uniformidad es lo que permite que un cliente genérico pueda navegar la API sin conocerla de antemano. La uniformidad también facilita la documentación automática con OpenAPI.

Del concepto a ASP.NET Core 8: Recurso y representación

REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Aplicación en el proyecto real. Este comportamiento está materializado en `Modelo/Orden.cs` del estado `2.1-diseno-apis-rest`. Busca la instrucción `Orden` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 3 — Representaciones y negociación de contenido

Un recurso es un concepto abstracto. Su representación es la forma concreta en la que se transmite. Un mismo recurso puede tener varias representaciones: JSON, XML, texto plano. La representación se selecciona mediante negociación de contenido. El cliente envía la cabecera Accept con los formatos que acepta. El servidor responde con el formato que puede generar. La negociación evita que el cliente tenga que especificar el formato en la URI. La representación JSON es la más común en APIs REST modernas. El curso usa JSON como representación por defecto.

```
// El cliente envía:
// Accept: application/json

app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes.AsNoTracking().ToListAsync());
```

- **Línea 1:** // El cliente envía: → comentario que indica el ejemplo.
- **Línea 2:** // Accept: application/json → cabecera del cliente.
- **Línea 3:** app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) => → declara el endpoint GET.

- **Línea 4:** await db.Ordenes.AsNoTracking().ToListAsync()); → devuelve la lista en JSON.

La representación tiene dos consecuencias prácticas. La primera es que la misma URI puede devolver formatos distintos según la cabecera Accept. La segunda es que el servidor puede devolver 406 Not Acceptable si no puede generar ningún formato solicitado. La negociación de contenido es una de las sub-restricciones de la interfaz uniforme. Su aplicación correcta evita que el cliente tenga que usar URIs distintas para formatos distintos.

Bloque 4 — Métodos HTTP y su semántica

Los métodos HTTP expresan la operación que se realiza sobre el recurso. Cada método tiene una semántica definida por el estándar. Los métodos principales son cinco: GET, POST, PUT, PATCH y DELETE. GET recupera una representación del recurso. POST crea un recurso nuevo. PUT reemplaza un recurso existente. PATCH modifica parcialmente un recurso existente. DELETE elimina un recurso existente. Cada método tiene dos propiedades importantes: idempotencia y seguridad. Un método es seguro si no modifica el estado del servidor. Un método es idempotente si ejecutarlo varias veces produce el mismo resultado que ejecutarlo una vez.

Métodos HTTP:

- **GET /api/v1/ordenes** → lista de órdenes. Seguro, idempotente.
- **GET /api/v1/ordenes/{id}** → orden concreta. Seguro, idempotente.
- **POST /api/v1/ordenes** → crear orden. No seguro, no idempotente.
- **PUT /api/v1/ordenes/{id}** → reemplazar orden. No seguro, idempotente.
- **PATCH /api/v1/ordenes/{id}** → modificar orden. No seguro, no idempotente.
- **DELETE /api/v1/ordenes/{id}** → eliminar orden. No seguro, idempotente.
- **Línea 1:** GET /api/v1/ordenes → recupera la colección. Seguro e idempotente.
- **Línea 2:** GET /api/v1/ordenes/{id} → recupera una orden. Seguro e idempotente.
- **Línea 3:** POST /api/v1/ordenes → crea una orden. No seguro, no idempotente.
- **Línea 4:** PUT /api/v1/ordenes/{id} → reemplaza una orden. No seguro, idempotente.
- **Línea 5:** PATCH /api/v1/ordenes/{id} → modifica parcialmente. No seguro, no idempotente.
- **Línea 6:** DELETE /api/v1/ordenes/{id} → elimina una orden. No seguro, idempotente.

La semántica de los métodos tiene dos consecuencias prácticas. La primera es que los métodos seguros pueden ser cacheados por intermediarios sin riesgo. La segunda es que los métodos idempotentes pueden ser reintentados sin riesgo de duplicación. La combinación de ambas propiedades determina qué métodos pueden reintentarse automáticamente. GET, PUT y DELETE pueden reintentarse. POST y PATCH no pueden reintentarse sin una clave de idempotencia. La semántica correcta de los métodos es la base de la comunicación fiable.

Del concepto a ASP.NET Core 8: DTO público

El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Aplicación en el proyecto real. Este comportamiento está materializado en `Modelo/OrdenDto.cs` del estado `2.1-diseno-apis-rest`. Busca la instrucción `OrdenDto` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el

recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Modelo/OrdenDto.cs`

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 5 — Códigos de estado HTTP

Los códigos de estado HTTP expresan el resultado de la operación. Se organizan en cinco familias: 1xx informativos, 2xx éxito, 3xx redirección, 4xx error del cliente, 5xx error del servidor. Cada familia tiene códigos específicos para situaciones concretas. Los códigos más usados en APIs REST son 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity y 500 Internal Server Error. La elección del código adecuado permite que el cliente distinga entre situaciones distintas sin analizar el cuerpo de la respuesta.

Códigos de estado habituales:

- **200 OK** → operación exitosa con cuerpo.
- **201 Created** → recurso creado, con cabecera Location.
- **204 No Content** → operación exitosa sin cuerpo.
- **400 Bad Request** → error de validación o formato.
- **401 Unauthorized** → falta autenticación.
- **403 Forbidden** → autenticación correcta, sin permisos.
- **404 Not Found** → recurso no encontrado.
- **409 Conflict** → conflicto de estado.
- **422 Unprocessable Entity** → entidad semánticamente inválida.
- **500 Internal Server Error** → error del servidor.
- **Línea 1:** 200 OK → operación exitosa con cuerpo.
- **Línea 2:** 201 Created → recurso creado, con cabecera Location.
- **Línea 3:** 204 No Content → operación exitosa sin cuerpo.
- **Línea 4:** 400 Bad Request → error de validación o formato.
- **Línea 5:** 401 Unauthorized → falta autenticación.
- **Línea 6:** 403 Forbidden → sin permisos.
- **Línea 7:** 404 Not Found → recurso no encontrado.
- **Línea 8:** 409 Conflict → conflicto de estado.
- **Línea 9:** 422 Unprocessable Entity → entidad semánticamente inválida.
- **Línea 10:** 500 Internal Server Error → error del servidor.

La elección del código de estado tiene dos consecuencias prácticas. La primera es que el cliente puede reaccionar de forma distinta a cada código sin analizar el cuerpo. La segunda es que los intermediarios pueden cachear o no cachear según el código. Los códigos 2xx son cacheables según las cabeceras. Los códigos 4xx y 5xx no son cacheables por defecto. La elección correcta del código es la base de la comunicación predecible entre cliente y servidor.

Bloque 6 — Diseño de URIs en APIs REST

El diseño de URIs sigue cinco reglas prácticas. La primera es usar sustantivos en plural para las colecciones. La segunda es usar el identificador del recurso concreto después de la colección. La tercera es anidar las relaciones con sub-rutas. La cuarta es no incluir verbos en las URIs. La quinta es versionar la API en la propia URI. La aplicación conjunta de las cinco reglas produce URIs predecibles y consistentes. La consistencia permite que un cliente genérico pueda navegar la API sin conocerla de antemano.

Reglas de diseño de URIs:

- **1. Sustantivos en plural** → /ordenes, /materiales.
- **2. Identificador tras colección** → /ordenes/{id}.
- **3. Anidamiento de relaciones** → /ordenes/{id}/lineas.
- **4. Sin verbos** → no /crearOrden.
- **5. Versionado en la URI** → /api/v1/ordenes.
- **Línea 1:** 1. Sustantivos en plural → colecciones con sustantivo plural.
- **Línea 2:** 2. Identificador tras colección → identificador después de la colección.
- **Línea 3:** 3. Anidamiento de relaciones → sub-rutas para relaciones.
- **Línea 4:** 4. Sin verbos → las URIs no contienen verbos.
- **Línea 5:** 5. Versionado en la URI → la versión va en la URI.

Las cinco reglas se aplican de forma uniforme en toda la API. La uniformidad es lo que distingue una API REST de una API RPC. La uniformidad también facilita la documentación automática con OpenAPI, la generación de clientes y la integración con herramientas de monitorización. La violación de cualquiera de las cinco reglas rompe la uniformidad y obliga al cliente a conocer la API de antemano. El curso aplica las cinco reglas al diseño de la API de la acería.

Del concepto a ASP.NET Core 8: Persistencia mediante EF Core

El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. Los tests conservan el proveedor SQL Server y cambian la conexión por la de una base aislada. El motor puede ser una instancia LocalDB configurada para pruebas o SQL Server en un contenedor administrado por Testcontainers. El estudio integrador de este punto explica ese recorrido.

Aplicación en el proyecto real. Este comportamiento está materializado en `Datos/AceriaDbContext.cs` del estado `2.1-diseno-apis-rest`. Busca la instrucción `AceriaDbContext` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 7 — Interfaz uniforme en ASP.NET Core

La interfaz uniforme se materializa en ASP.NET Core con cuatro elementos. El primero es el enrutamiento, que asocia URIs a manejadores. El segundo es el enlace de parámetros, que asocia partes de la petición a parámetros del manejador. El tercero es la serialización, que convierte objetos a JSON y viceversa. El cuarto es la negociación de contenido, que selecciona el formato de la respuesta. Los cuatro elementos se configuran en el arranque de la aplicación. La configuración correcta produce una API uniforme sin código repetido en cada endpoint.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes.AsNoTracking().ToListAsync());

app.MapGet("/api/v1/ordenes/{id:guid}", async (Guid id, AceriaDbContext db) =>
{
    var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);
    return orden is not null ? Results.Ok(orden) : Results.NotFound();
});

app.Run();
```

- **Línea 1:** var builder = WebApplication.CreateBuilder(args); → crea el constructor de la aplicación.
- **Línea 2:** builder.Services.AddEndpointsApiExplorer(); → habilita la exploración de endpoints.
- **Línea 3:** builder.Services.AddSwaggerGen(); → registra el generador de Swagger.
- **Línea 4:** var app = builder.Build(); → construye la aplicación.
- **Línea 5:** app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) => → declara el endpoint de colección.
- **Línea 6:** await db.Ordenes.AsNoTracking().ToListAsync()); → devuelve la lista.
- **Línea 7:** app.MapGet("/api/v1/ordenes/{id:guid}", async (Guid id, AceriaDbContext db) => → declara el endpoint de detalle.
- **Línea 8:** { → abre el cuerpo del lambda.
- **Línea 9:** var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id); → consulta la orden.
- **Línea 10:** return orden is not null ? Results.Ok(orden) : Results.NotFound(); → devuelve 200 o 404.
- **Línea 11:** }); → cierra el lambda.
- **Línea 12:** app.Run(); → arranca el servidor.

La configuración de la interfaz uniforme tiene dos consecuencias prácticas. La primera es que los endpoints siguen un patrón común que facilita la lectura. La segunda es que la documentación OpenAPI se genera automáticamente a partir de los endpoints. La combinación de ambas reduce el coste de mantenimiento y mejora la experiencia del consumidor. La interfaz uniforme es una de las propiedades más valiosas de REST.

Bloque 8 — Recursos anidados y relaciones

Los recursos pueden anidarse para expresar relaciones. Una orden tiene líneas. Una línea tiene materiales. Las relaciones se expresan con sub-rutas. La URI /api/v1/ordenes/{id}/lineas expresa la colección de líneas de una orden concreta. La URI /api/v1/ordenes/{id}/lineas/{lineaId} expresa una línea concreta. El anidamiento tiene un límite práctico: no se anidan más de dos niveles para evitar

URIs excesivamente largas. Las relaciones que superan dos niveles se expresan con recursos independientes y parámetros de consulta.

Anidamiento de recursos:

- **/api/v1/ordenes** → colección de órdenes.
- **/api/v1/ordenes/{id}** → orden concreta.
- **/api/v1/ordenes/{id}/lineas** → líneas de una orden.
- **/api/v1/ordenes/{id}/lineas/{lid}** → línea concreta.

Alternativa sin anidar:

- **/api/v1/lineas?ordenId={id}** → líneas filtradas por orden.
- **Línea 1:** /api/v1/ordenes → colección de órdenes.
- **Línea 2:** /api/v1/ordenes/{id} → orden concreta.
- **Línea 3:** /api/v1/ordenes/{id}/lineas → líneas de una orden.
- **Línea 4:** /api/v1/ordenes/{id}/lineas/{lid} → línea concreta.
- **Línea 5:** /api/v1/lineas?ordenId={id} → alternativa sin anidar.

El anidamiento tiene dos consecuencias prácticas. La primera es que las relaciones se expresan de forma natural. La segunda es que las URIs pueden volverse largas si se anidan demasiados niveles. La regla práctica es anidar solo cuando la relación es de composición: la línea no existe sin la orden. Cuando la relación es de asociación, se usa un recurso independiente con parámetros de consulta. La distinción entre composición y asociación es la clave para decidir cuándo anidar.

Del concepto a ASP.NET Core 8: Endpoint GET de colección

El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.1-diseno-apis-rest`. Busca la instrucción `MapGet` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Program.cs`

```
// INICIO PASO 2.1.8: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")

;
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 9 — Paginación, filtrado y ordenación

Las colecciones grandes no se devuelven completas. Se paginan con parámetros de consulta. Los parámetros habituales son page y pageSize para la paginación, filter para el filtrado y sort para la ordenación. Los parámetros de consulta se expresan con el signo ? después de la URI. La respuesta incluye metadatos de paginación: número de página, tamaño de página, total de elementos y total de páginas. Los metadatos permiten al cliente construir la interfaz de navegación sin conocer la colección completa.

Paginación, filtrado y ordenación:

GET /api/v1/ordenes?page=1&pageSize=20
GET /api/v1/ordenes?filter=cliente:ACX-001
GET /api/v1/ordenes?sort=fechaCreacion:desc

Respuesta con metadatos:

```
{
  "items": [ ... ],
  "page": 1,
  "pageSize": 20,
  "totalItems": 145,
  "totalPages": 8
}
```

- **Línea 1:** GET /api/v1/ordenes?page=1&pageSize=20 → paginación.
- **Línea 2:** GET /api/v1/ordenes?filter=cliente:ACX-001 → filtrado.
- **Línea 3:** GET /api/v1/ordenes?sort=fechaCreacion:desc → ordenación.
- **Línea 4:** { → abre el objeto de respuesta.
- **Línea 5:** "items": [...], → lista de elementos de la página.
- **Línea 6:** "page": 1, → número de página actual.
- **Línea 7:** "pageSize": 20, → tamaño de página.
- **Línea 8:** "totalItems": 145, → total de elementos.
- **Línea 9:** "totalPages": 8 → total de páginas.

La paginación tiene dos consecuencias prácticas. La primera es que las respuestas se mantienen pequeñas y predecibles. La segunda es que el cliente puede construir la interfaz de navegación con los metadatos. La ausencia de paginación en colecciones grandes produce respuestas lentas, consumo elevado de memoria y dificultad para el cliente. La paginación es una buena práctica obligatoria en cualquier API REST que exponga colecciones.

Bloque 10 — Aplicación del diseño REST al proyecto del curso

El proyecto del curso aplica el diseño REST en cada endpoint. Los recursos se identifican con URIs que siguen las cinco reglas. Los métodos HTTP expresan la semántica de la operación. Los códigos de estado expresan el resultado. Las representaciones se negocian con la cabecera Accept. Las colecciones se paginan con parámetros de consulta. Las relaciones se anidan con sub-rutas. El diseño se documenta en un documento de diseño de APIs REST y se audita con una clase de validación de URIs.

Aplicación al proyecto del curso:

- **Recursos identificados** → /api/v1/ordenes, /api/v1/materiales.
- **Métodos HTTP con semántica** → GET, POST, PUT, PATCH, DELETE.
- **Códigos de estado** → 200, 201, 204, 400, 404, 409, 422, 500.

- **Representaciones** → JSON por defecto.
- **Paginación** → page, pageSize.
- **Relaciones anidadas** → /ordenes/{id}/lineas.
- **Línea 1:** Recursos identificados → URIs que siguen las cinco reglas.
- **Línea 2:** Métodos HTTP con semántica → métodos con semántica definida.
- **Línea 3:** Códigos de estado → códigos adecuados a cada situación.
- **Línea 4:** Representaciones → JSON por defecto.
- **Línea 5:** Paginación → parámetros de consulta.
- **Línea 6:** Relaciones anidadas → sub-rutas para relaciones.

La aplicación del diseño REST prepara el proyecto para los puntos posteriores del módulo 2. El punto 2.2 estudiará los recursos, verbos y códigos con detalle. El punto 2.3 estudiará la validación de peticiones. El punto 2.4 estudiará la documentación OpenAPI. La base del diseño REST es el fundamento de los tres puntos siguientes. El punto 2.1 establece los conceptos que se aplicarán en los tres puntos siguientes.

Resumen rápido de la teoría

- REST es un estilo arquitectónico definido por seis restricciones: cliente-servidor, sin estado, cacheable, interfaz uniforme, sistema en capas y código bajo demanda.
- Los recursos son la unidad central del diseño y se identifican con URIs que siguen cinco reglas.
- Las representaciones se negocian con la cabecera Accept y el formato por defecto es JSON.
- Los métodos HTTP expresan la semántica de la operación y tienen dos propiedades: seguridad e idempotencia.
- Los códigos de estado expresan el resultado de la operación en cinco familias.
- Las colecciones se paginan con parámetros de consulta y las relaciones se anidan con sub-rutas.

Del concepto a ASP.NET Core 8: GET individual y pruebas HTTP

La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.1-diseno-apis-rest`. Busca la instrucción `ObtenerOrdenPorId` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Estudio integrador 2.1 — Probar HTTP y persistencia con SQL Server real

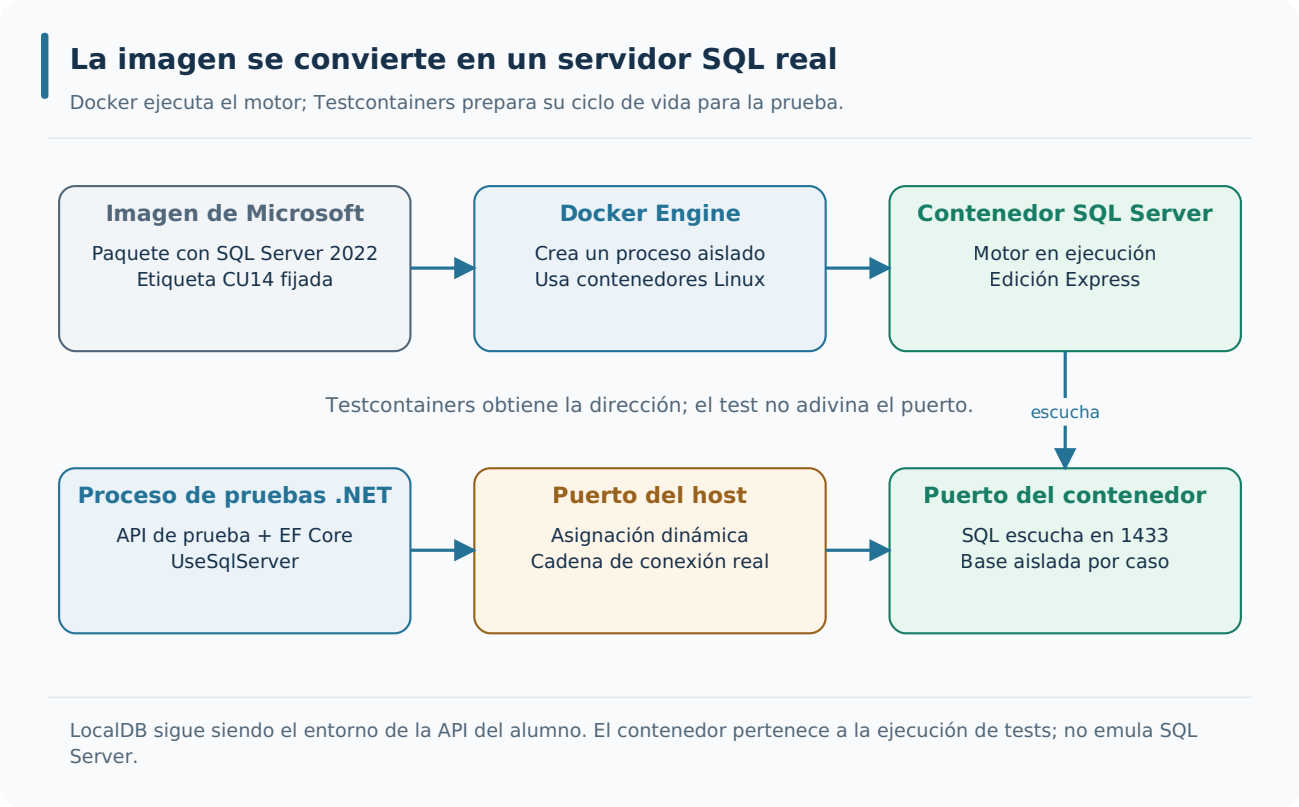
Por qué el proveedor forma parte del comportamiento

Una prueba que envía un POST y después recupera una orden mediante GET atraviesa varias piezas: enrutamiento, deserialización JSON, validación, contexto de EF Core y almacenamiento. Si cambia la pieza de almacenamiento, también puede cambiar lo que demuestra el resultado. En este curso la API usa `Microsoft.EntityFrameworkCore.SqlServer`; sus pruebas de integración conservan ese proveedor y ejecutan el SQL contra un motor SQL Server real.

EF Core InMemory guarda datos sin ser un motor relacional. SQLite sí es relacional, pero tiene su propio dialecto, tipos y reglas de ejecución. Ninguno demuestra por sí solo que una consulta o restricción funcionará igual en SQL Server. Los dobles siguen siendo útiles para comprobar una regla de dominio o la coordinación de un caso de uso. La pregunta «¿se guardan y recuperan correctamente estos datos con nuestro proveedor?» necesita una prueba que llegue al motor elegido. Microsoft explica estas diferencias al comparar estrategias de prueba de EF Core.

Cuatro conceptos de Docker sin cambiar la aplicación del alumno

Docker permite ejecutar procesos en entornos aislados y gestionar sus archivos, red y ciclo de vida. **Una imagen** contiene el programa y los archivos necesarios para arrancarlo. **Un contenedor** es una instancia en ejecución creada a partir de esa imagen. **Un puerto publicado** enlaza un puerto accesible desde el proceso de pruebas con el puerto donde escucha el servidor dentro del contenedor. La imagen es el paquete; el contenedor ejecuta el programa.



Interpretación del gráfico: La fila superior explica cómo aparece el servidor. La inferior muestra por dónde llega EF Core hasta él. El contenedor ejecuta SQL Server; Testcontainers no convierte SQL Server en una simulación. La figura también está disponible en PNG en la misma carpeta.

Para el curso se fija la imagen `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04` y la variable `MSSQL_PID=Express`. La etiqueta identifica la versión utilizada por el laboratorio; no significa «la versión más reciente». La edición Express se configura dentro del contenedor. LocalDB es otra forma de ejecutar SQL Server Express, orientada al desarrollo en Windows: no es el programa que arranca dentro de esta imagen Linux.

La aplicación que el alumno abre en Visual Studio continúa utilizando LocalDB. Las pruebas pueden iniciar el contenedor con Docker Desktop en modo de contenedores Linux. Esta separación evita tener que modificar la conexión cotidiana de la API. El objetivo aquí es disponer de una dependencia real y desechable para probar persistencia, no empaquetar ni desplegar la API en Docker.

Qué automatiza Testcontainers y qué escribe el test

`Testcontainers.MsSql` es una biblioteca de .NET. Habla con Docker, prepara un contenedor de SQL Server, espera a que esté disponible y permite obtener su cadena de conexión. La versión fijada en el proyecto es **3.10.0**. El helper de pruebas `BaseSqlServer` encapsula esa preparación y el aislamiento de las bases. Conviene distinguirlo del `DbContext`: el helper consigue un destino de pruebas; el contexto mapea entidades y consulta ese destino.

Este fragmento explicativo resume la rama Docker del helper. Su código completo y su limpieza se construyen en las prácticas; no es un archivo adicional que deba copiarse sobre ellas.

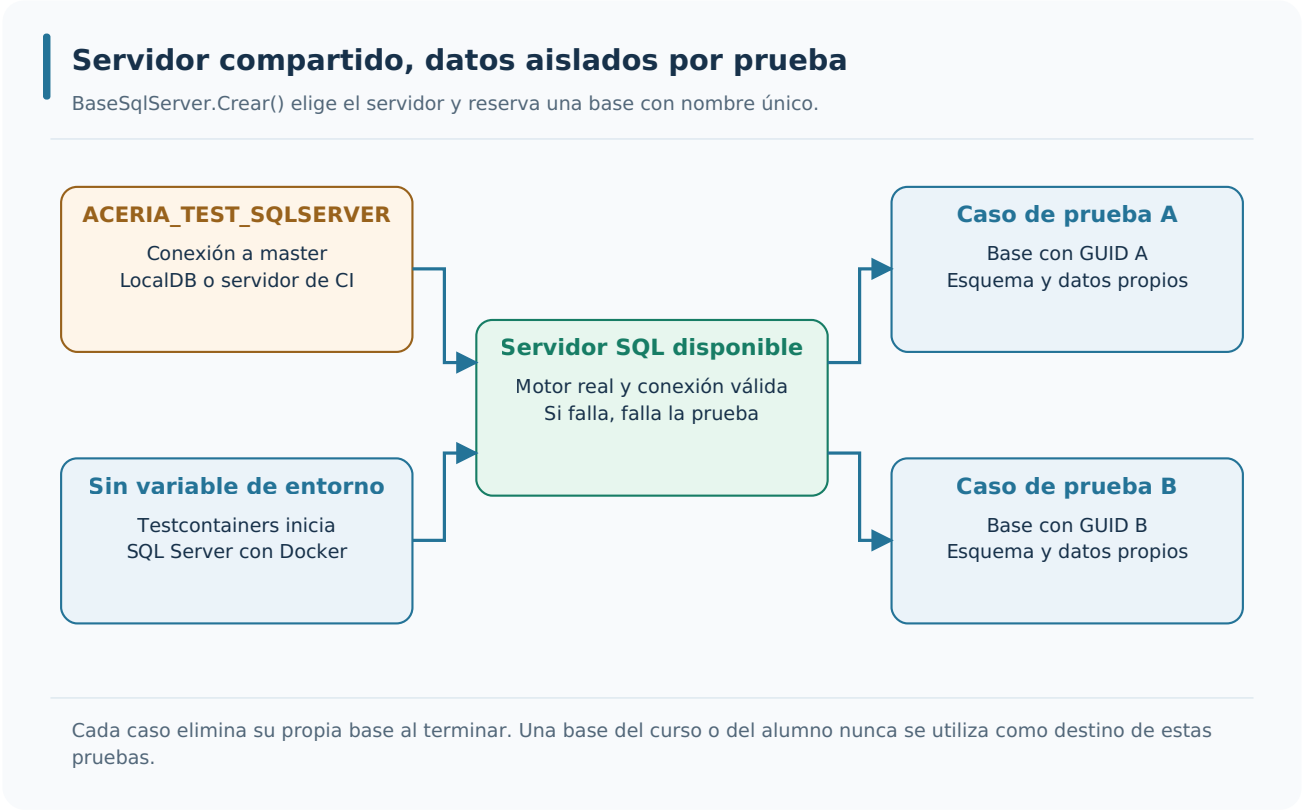
```
var contenedor = new MsSqlBuilder()
    .WithImage("mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04")
    .WithEnvironment("MSSQL_PID", "Express")
    .Build();
await contenedor.StartAsync();
var cadenaServidor = contenedor.GetConnectionString();
```

- **Línea 1:** `MsSqlBuilder` prepara la configuración de un contenedor SQL Server. Todavía no hay un servidor ejecutándose.
- **Línea 2:** `WithImage` fija la imagen del laboratorio. Así, una ejecución posterior no adopta otra etiqueta por usar `latest`.
- **Línea 3:** `MSSQL_PID` selecciona la edición Express del motor.
- **Línea 4:** `Build` crea el objeto con el que se gestionará el contenedor; no equivale a arrancarlo.
- **Línea 5:** `StartAsync` inicia el recurso y espera su disponibilidad. Si no hay un Docker utilizable o el motor no arranca, la preparación debe fallar.
- **Línea 6:** `GetConnectionString` devuelve la dirección, el puerto publicado y las credenciales del servidor de pruebas. El test no debe inventar una conexión fija a `localhost,1433`.

El puerto **1433** pertenece al servidor dentro del contenedor. Docker puede publicarlo en otro puerto libre del host. Dos ejecuciones no tienen que competir por un único puerto del ordenador. El alumno usa la cadena devuelta por el helper; modificar el puerto a mano rompe esa separación. La documentación de Testcontainers detalla cómo obtener las direcciones y puertos asignados.

La variable de entorno es una alternativa de servidor

`BaseSqlServer.Crear()` consulta `ACERIA_TEST_SQLSERVER`. Si existe, la interpreta como una conexión a `master` en un SQL Server disponible. Si no existe, usa Testcontainers. En ambos caminos crea una base de datos con nombre único basado en GUID y entrega la cadena de esa base mediante `ConnectionString`. La cuenta de pruebas necesita permisos para crear y eliminar sus bases aisladas. El helper no debe apuntar a una base de trabajo del alumno para vaciarla.



Interpretación del gráfico: Se puede compartir el proceso servidor sin compartir las filas de cada caso. La variable de entorno cambia cómo se obtiene el servidor; el contrato de aislamiento de `BaseSqlServer` permanece.

En PowerShell, esta alternativa usa la instancia LocalDB instalada en Windows:

```
$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)
\MSSQLLocalDB;Database=master;Trusted_Connection=True;TrustServerCertificate=True'
dotnet test
Remove-Item Env:ACERIA_TEST_SQLSERVER
```

- **Línea 1:** Define la conexión del servidor de pruebas en la sesión actual. `master` es el punto de entrada administrativo; los datos del caso se guardarán en otra base con nombre único.
- **Línea 2:** Ejecuta las pruebas de la solución situada en el directorio actual. La factoría utiliza la base aislada creada por el helper.
- **Línea 3:** Elimina la variable de esa sesión. La siguiente ejecución vuelve a la rama Docker si no recibe otra configuración.

`TrustServerCertificate=True` corresponde aquí a una conexión de laboratorio. No describe por sí sola una política de certificados para producción. La alternativa LocalDB permite ejecutar estas pruebas en Windows sin arrancar Docker, pero sigue requiriendo un motor SQL Server operativo. Una configuración inválida, un servicio detenido o una descarga fallida no se transforman en pruebas omitidas ni activan InMemory como sustituto.

Recorrido de una prueba desde la factoría hasta la limpieza

La factoría hereda de `WebApplicationFactory<Program>`. Construye un host de ASP.NET Core para enviar solicitudes mediante `HttpClient`; no necesita abrir el navegador. Al configurar servicios reemplaza la conexión del contexto. El esquema se crea sobre la base de pruebas una vez aplicada esa configuración, nunca sobre la base local de la aplicación por accidente.

```
private readonly BaseSqlServer _base = BaseSqlServer.Crear();
services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer(_base.ConnectionString));
```

- **Línea 1:** Reserva una base aislada mediante el helper del namespace `AceriaApp.PruebasSqlServer`. Su duración queda vinculada a la factoría.
- **Línea 2:** Registra el contexto que recibirán los endpoints mediante inyección de dependencias. En la factoría se retira antes el registro de opciones que traía la aplicación.
- **Línea 3:** Mantiene el proveedor `UseSqlServer` y cambia únicamente la conexión. El test ejercita traducción SQL, restricciones y persistencia del motor utilizado por el curso.

Momento	Qué sucede	Qué permite comprobar
Preparar	Obtener servidor y crear base con GUID	El caso empieza sin filas de otras pruebas.
Componer	Configurar `UseSqlServer` y crear esquema	Los endpoints reciben el contexto correcto.
Actuar	Enviar POST con datos válidos	La ruta y la escritura se ejecutan de extremo a extremo.
Observar	Enviar GET o abrir otro contexto	El resultado procede de datos persistidos y no solo del objeto recién creado.
Terminar	Liberar cliente, host y contextos; eliminar base	El siguiente caso no hereda datos ni conexiones del anterior.

El aislamiento tiene un coste: arrancar SQL Server tarda más que crear una lista en memoria. La primera ejecución también puede descargar la imagen. Ese tiempo compra una comprobación distinta y necesaria. Una prueba del dominio que no usa persistencia puede seguir ejecutándose sin SQL; una prueba de integración que lo necesita debe demostrarlo con SQL disponible. Un resultado satisfactorio verifica los escenarios ejecutados, no certifica por sí solo carga, alta disponibilidad ni todas las diferencias entre versiones del servidor.

Comprobación de comprensión. Antes de continuar con los verbos HTTP, explica con tus palabras por qué imagen, contenedor y base de datos no son sinónimos; señala dónde cambia la conexión del test; y describe qué dato nuevo debería observar un GET después de un POST. Esa explicación enlaza el contrato REST con la persistencia que lo sostiene.

Documentación de consulta de este estudio

- Imágenes de contenedor y publicación de puertos: paquete, proceso y acceso desde el host.
- Módulo SQL Server de Testcontainers: configuración y ciclo de vida. Contrasta los ejemplos actuales con la versión 3.10.0 fijada en el curso.
- Variables de SQL Server en Linux: significado de `MSSQL_PID`.
- SQL Server Express LocalDB: entorno de desarrollo local en Windows.

Punto 2.2 — Recursos, verbos HTTP y códigos de respuesta

Módulo 2: Desarrollo de APIs REST con ASP.NET Core Objetivos de aprendizaje:

- Comprender el concepto de recurso y su representación en una API REST.
- Aplicar los verbos HTTP con su semántica correcta.

- Seleccionar el código de respuesta adecuado a cada situación.
- Comprender la relación entre verbo, código de respuesta y estado del recurso.
- Añadir al proyecto AceriaApp una clase de validación de combinaciones verbo-código y un catálogo de recursos del dominio.

Bloque 1 — El recurso como unidad de diseño

Un recurso es una entidad identificable que puede ser referenciada y manipulada a través de una API. No es lo mismo que una entidad de base de datos ni que una clase del dominio. Un recurso es una abstracción que agrupa los datos y las operaciones que el cliente puede realizar sobre ellos. Un recurso puede corresponder a una fila de una tabla, a una vista que combina varias tablas o a un agregado de negocio. La decisión de qué constituye un recurso se toma desde la perspectiva del cliente, no desde la perspectiva del modelo de datos. Un recurso bien definido expone solo los datos que el cliente necesita y solo las operaciones que el cliente puede realizar.

Recursos del dominio de la acería:

- **Orden** → orden de producción.
- **LineaOrden** → línea de una orden.
- **Material** → materia prima.
- **Cliente** → empresa compradora.
- **Movimiento** → movimiento de inventario.
- **Línea 1:** Orden → recurso que representa una orden de producción.
- **Línea 2:** LineaOrden → recurso que representa una línea de una orden.
- **Línea 3:** Material → recurso que representa una materia prima.
- **Línea 4:** Cliente → recurso que representa una empresa compradora.
- **Línea 5:** Movimiento → recurso que representa un movimiento de inventario.

Cada recurso tiene una representación por defecto y puede tener representaciones alternativas. La representación por defecto es la que el servidor devuelve cuando el cliente no especifica ninguna preferencia. La representación alternativa se negocia con la cabecera Accept. El recurso también tiene un estado interno que el servidor gestiona, pero que no se expone directamente al cliente. El cliente solo ve la representación, no el estado. Esta separación entre recurso y representación es una de las propiedades más importantes del estilo REST y la que permite que el mismo recurso se consuma desde clientes con formatos distintos.

La identificación de recursos del dominio de la acería sigue un criterio de negocio, no de base de datos. La tabla Ordenes de la base de datos puede contener columnas internas que no forman parte del recurso Orden. La columna IdempotencyKey, por ejemplo, es un detalle de implementación que no se expone como parte del recurso. Del mismo modo, el recurso Orden puede combinar datos de varias tablas para formar una representación coherente desde la perspectiva del cliente. El recurso es una proyección diseñada para el consumo, no un reflejo de la estructura de persistencia.

Bloque 2 — Representaciones del recurso

Una representación es la forma concreta en la que un recurso se transmite entre cliente y servidor. La representación por defecto en las APIs REST modernas es JSON. JSON es un formato de texto ligero, legible y ampliamente soportado. Otras representaciones posibles son XML, YAML, CSV o binario. La representación se selecciona mediante negociación de contenido. El cliente envía la cabecera Accept con los formatos que acepta, y el servidor responde con el formato que puede generar. Si el servidor no puede generar ninguno de los formatos solicitados, responde con el código 406 Not Acceptable.

```
public record OrdenRepresentacion(
    Guid Id,
    string Cliente,
    string Material,
    int Cantidad,
    DateTime FechaCreacion);
```

- **Línea 1:** public record OrdenRepresentacion(→ declara la representación de una orden.
- **Línea 2:** Guid Id, → identificador de la orden.
- **Línea 3:** string Cliente, → nombre del cliente.
- **Línea 4:** string Material, → material solicitado.
- **Línea 5:** int Cantidad, → cantidad solicitada.
- **Línea 6:** DateTime FechaCreacion); → fecha de creación.

La representación no contiene toda la información del recurso: contiene solo la información que el cliente necesita. Si el recurso Orden tiene veinte propiedades internas, la representación puede exponer solo cinco. La selección de qué propiedades se exponen se realiza en función de los casos de uso del cliente. Una representación de colección puede exponer menos propiedades que una representación de detalle. La distinción entre representación de colección y representación de detalle es una práctica habitual para reducir el tamaño de las respuestas y mejorar el rendimiento.

Del concepto a ASP.NET Core 8: Contrato de entrada

El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Aplicación en el proyecto real. Este comportamiento está materializado en `Contratos/ContratosOrden.cs` del estado `2.2-recursos-verbos-codigos`. Busca la instrucción `CrearOrdenRequest` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 3 — Verbos HTTP y su semántica completa

Los verbos HTTP tienen una semántica definida por el estándar y por las convenciones del ecosistema REST. El verbo GET recupera una representación del recurso. El verbo POST crea un recurso nuevo. El verbo PUT reemplaza un recurso existente. El verbo PATCH modifica parcialmente un recurso existente. El verbo DELETE elimina un recurso existente. Cada verbo tiene tres propiedades: seguridad, idempotencia y cacheabilidad. La seguridad indica si el verbo modifica el estado del servidor. La idempotencia indica si ejecutar el verbo varias veces produce el mismo resultado que ejecutarlo una vez. La cacheabilidad indica si la respuesta puede almacenarse en caché.



Figura M02.2.2.3 — Cómo relacionar verbos HTTP con resultados.

Propiedades de los verbos:

Verbo	Seguro	Idempotente	Cacheable
GET	Sí	Sí	Sí
POST	No	No	No
PUT	No	Sí	No
PATCH	No	No	No
DELETE	No	Sí	No

- **Línea 1:** GET → seguro, idempotente, cacheable.
- **Línea 2:** POST → no seguro, no idempotente, no cacheable.
- **Línea 3:** PUT → no seguro, idempotente, no cacheable.
- **Línea 4:** PATCH → no seguro, no idempotente, no cacheable.
- **Línea 5:** DELETE → no seguro, idempotente, no cacheable.

La elección del verbo correcto no es una cuestión de preferencia: es una cuestión de semántica. Si se usa POST para reemplazar un recurso, se pierde la idempotencia y los reintentos pueden duplicar la operación. Si se usa GET para crear un recurso, se pierde la seguridad y los rastreadores pueden crear recursos sin querer. La semántica correcta de los verbos es la base de la comunicación fiable entre cliente y servidor. Un endpoint que usa el verbo incorrecto es un endpoint que falla en producción de formas difíciles de diagnosticar.

Bloque 4 — Verbos en colecciones y en recursos concretos

La combinación de verbo y URI define la operación. Los verbos se aplican de forma distinta a colecciones y a recursos concretos. Sobre una colección, GET devuelve la lista, POST crea un elemento nuevo y DELETE elimina la colección completa. Sobre un recurso concreto, GET devuelve el recurso, PUT lo reemplaza, PATCH lo modifica parcialmente y DELETE lo elimina. La combinación de verbo y URI produce operaciones con semántica clara y predecible. La previsibilidad permite que el cliente sepa qué esperar sin consultar la documentación.

Combinaciones verbo-URI:

- **GET /api/v1/ordenes** → lista de órdenes.
- **POST /api/v1/ordenes** → crear orden.

- **GET /api/v1/ordenes/{id}** → orden concreta.
- **PUT /api/v1/ordenes/{id}** → reemplazar orden.
- **PATCH /api/v1/ordenes/{id}** → modificar orden.
- **DELETE /api/v1/ordenes/{id}** → eliminar orden.
- **Línea 1:** GET /api/v1/ordenes → recupera la colección.
- **Línea 2:** POST /api/v1/ordenes → crea un elemento nuevo.
- **Línea 3:** GET /api/v1/ordenes/{id} → recupera un recurso concreto.
- **Línea 4:** PUT /api/v1/ordenes/{id} → reemplaza el recurso concreto.
- **Línea 5:** PATCH /api/v1/ordenes/{id} → modifica parcialmente el recurso.
- **Línea 6:** DELETE /api/v1/ordenes/{id} → elimina el recurso concreto.

La combinación de verbo y URI también determina el código de respuesta adecuado. GET sobre una colección devuelve 200 con la lista. POST sobre una colección devuelve 201 con la cabecera Location y el recurso creado. GET sobre un recurso concreto devuelve 200 o 404. PUT y PATCH devuelven 200 con el recurso actualizado o 204 sin cuerpo. DELETE devuelve 204 o 404. La relación entre verbo, URI y código de respuesta es sistemática y predecible.

Del concepto a ASP.NET Core 8: POST y creación

MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.2-recursos-verbos-codigos`. Busca la instrucción `MapPost` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Program.cs`

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 5 — Códigos de respuesta y su relación con el verbo

Los códigos de respuesta expresan el resultado de la operación. La relación entre verbo y código de respuesta es sistemática. GET devuelve 200 si encuentra el recurso y 404 si no lo encuentra. POST devuelve 201 si crea el recurso y 400 si la petición es inválida. PUT devuelve 200 o 204 si reemplaza el recurso y 404 si no existe. PATCH devuelve 200 o 204 si modifica el recurso y 404 si no existe. DELETE devuelve 204 si elimina el recurso y 404 si no existe. La sistematicidad de la relación permite al cliente anticipar el código de respuesta.

Relación verbo-código:

- **GET /ordenes** → 200 OK
- **POST /ordenes** → 201 Created
- **GET /ordenes/{id}** → 200 OK | 404 Not Found
- **PUT /ordenes/{id}** → 200 OK | 204 No Content | 404 Not Found
- **PATCH /ordenes/{id}** → 200 OK | 204 No Content | 404 Not Found
- **DELETE /ordenes/{id}** → 204 No Content | 404 Not Found
- **Línea 1:** GET /ordenes → 200 OK → recuperación exitosa de la colección.
- **Línea 2:** POST /ordenes → 201 Created → creación exitosa del recurso.
- **Línea 3:** GET /ordenes/{id} → 200 OK | 404 → recuperación exitosa o recurso no encontrado.
- **Línea 4:** PUT /ordenes/{id} → 200 | 204 | 404 → reemplazo exitoso o recurso no encontrado.
- **Línea 5:** PATCH /ordenes/{id} → 200 | 204 | 404 → modificación exitosa o recurso no encontrado.
- **Línea 6:** DELETE /ordenes/{id} → 204 | 404 → eliminación exitosa o recurso no encontrado.

La elección entre 200 y 204 depende de si la respuesta incluye un cuerpo. 200 indica éxito con cuerpo. 204 indica éxito sin cuerpo. En un PUT o PATCH, si el servidor devuelve el recurso actualizado, usa 200. Si no devuelve nada, usa 204. En un DELETE, el servidor usa 204 porque el recurso ya no existe y no hay cuerpo que devolver. La elección entre 200 y 204 es una decisión de diseño que se documenta y se mantiene consistente en toda la API.

Bloque 6 — Códigos de error del cliente

Los códigos de error del cliente se agrupan en la familia 4xx. Los más usados son 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 409 Conflict y 422 Unprocessable Entity. El código 400 indica que la petición está mal formada o tiene un error de sintaxis. El código 401 indica que falta autenticación. El código 403 indica que la autenticación es correcta pero el cliente no tiene permisos. El código 404 indica que el recurso no existe. El código 405 indica que el verbo no está permitido sobre el recurso. El código 409 indica un conflicto de estado. El código 422 indica que la entidad es sintácticamente correcta pero semánticamente inválida.

Códigos de error del cliente:

- **400 Bad Request** → petición mal formada.
- **401 Unauthorized** → falta autenticación.
- **403 Forbidden** → sin permisos.
- **404 Not Found** → recurso no encontrado.
- **405 Method Not Allowed** → verbo no permitido.
- **409 Conflict** → conflicto de estado.
- **422 Unprocessable Entity** → entidad semánticamente inválida.
- **Línea 1:** 400 Bad Request → petición mal formada.

- **Línea 2:** 401 Unauthorized → falta autenticación.
- **Línea 3:** 403 Forbidden → autenticación correcta, sin permisos.
- **Línea 4:** 404 Not Found → recurso no encontrado.
- **Línea 5:** 405 Method Not Allowed → verbo no permitido.
- **Línea 6:** 409 Conflict → conflicto de estado.
- **Línea 7:** 422 Unprocessable Entity → entidad semánticamente inválida.

La distinción entre 400 y 422 es una de las más sutiles. El código 400 se usa cuando la petición no se puede procesar por un error de formato o de sintaxis. El código 422 se usa cuando la petición se puede procesar pero la entidad no cumple las reglas de negocio. Un JSON mal formado produce 400. Un JSON bien formado con un campo Cantidad negativo produce 422. La distinción permite al cliente distinguir entre un error de programación y un error de datos. La distinción también permite a las herramientas de monitorización separar los errores de infraestructura de los errores de negocio.

Del concepto a ASP.NET Core 8: PUT y sustitución

El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.2-recursos-verbos-codigos`. Busca la instrucción `MapPut` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 7 — Códigos de error del servidor

Los códigos de error del servidor se agrupan en la familia 5xx. Los más usados son 500 Internal Server Error, 501 Not Implemented, 502 Bad Gateway, 503 Service Unavailable y 504 Gateway Timeout. El código 500 indica un error no controlado del servidor. El código 501 indica que el servidor no implementa la funcionalidad solicitada. El código 502 indica que un servidor intermedio recibió una respuesta inválida del servidor destino. El código 503 indica que el servicio no está disponible temporalmente. El código 504 indica que un servidor intermedio no recibió respuesta del servidor destino a tiempo.

Códigos de error del servidor:

- **500 Internal Server Error** → error no controlado.
- **501 Not Implemented** → funcionalidad no implementada.
- **502 Bad Gateway** → respuesta inválida del servidor destino.
- **503 Service Unavailable** → servicio no disponible.
- **504 Gateway Timeout** → timeout en servidor intermedio.
- **Línea 1:** 500 Internal Server Error → error no controlado.
- **Línea 2:** 501 Not Implemented → funcionalidad no implementada.
- **Línea 3:** 502 Bad Gateway → respuesta inválida del servidor destino.
- **Línea 4:** 503 Service Unavailable → servicio no disponible.

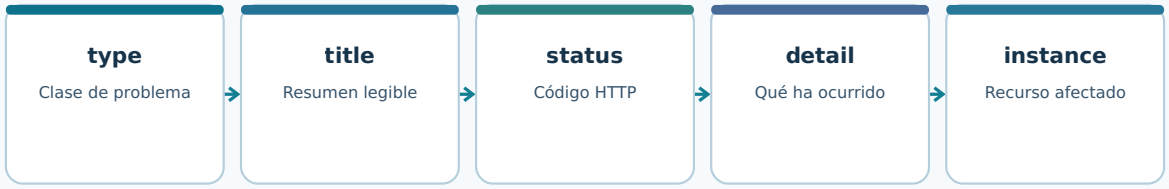
- **Línea 5:** 504 Gateway Timeout → timeout en servidor intermedio.

La elección del código 5xx adecuado es importante para la monitorización y para la comunicación con el cliente. Un 500 indica un error del servidor que requiere investigación. Un 503 indica que el servicio no está disponible temporalmente y que el cliente puede reintentar más tarde. Un 504 indica que un servidor intermedio no recibió respuesta a tiempo. La distinción permite a los sistemas de monitorización clasificar los errores y generar alertas específicas para cada tipo. La práctica de devolver siempre 500 para cualquier error del servidor oculta información valiosa.

Bloque 8 — Problem details como formato de error

El formato problem details está definido en la RFC 7807 y describe la estructura de un error en una API REST. Los campos principales son type, title, status, detail e instance. El campo type identifica el tipo de error con una URI. El campo title es un resumen legible del error. El campo status es el código HTTP. El campo detail explica el error concreto. El campo instance identifica la instancia concreta del error. El formato se usa en todas las respuestas de error para que el cliente pueda procesarlas de forma uniforme.

Un contrato común para las respuestas de error



La flecha indica una relación de diseño o una secuencia de trabajo.

Figura M02.2.2.8 — Los campos principales de Problem Details.

```
{
  "type": "https://acería.local/errores/validacion",
  "title": "Error de validación",
  "status": 400,
  "detail": "El campo Cantidad debe ser mayor que cero.",
  "instance": "/api/v1/ordenes"
}
```

- **Línea 1:** { → abre el objeto de error.
- **Línea 2:** "type": "https://acería.local/errores/validacion", → identifica el tipo de error.
- **Línea 3:** "title": "Error de validación", → resumen legible del error.
- **Línea 4:** "status": 400, → código HTTP del error.
- **Línea 5:** "detail": "El campo Cantidad debe ser mayor que cero.", → detalle del error.
- **Línea 6:** "instance": "/api/v1/ordenes" → instancia concreta del error.
- **Línea 7:** } → cierra el objeto de error.

El formato problem details se implementa en ASP.NET Core con la clase Results.Problem. La clase acepta los campos title, detail, statusCode y otros opcionales. La implementación por defecto usa el tipo about:blank cuando no se especifica. La adopción del formato evita la duplicación de estructuras

de error y facilita la integración con herramientas de monitorización. El formato se usa en todos los endpoints que devuelven errores.

Del concepto a ASP.NET Core 8: PATCH especializado

El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.2-recursos-verbos-codigos`. Busca la instrucción `MapPatch` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Program.cs`

```
// INICIO PASO 2.2.8: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad<=0) return Results.BadRequest(new {error="Cantidad debe ser positiva."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")


;
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 9 — Catálogo de recursos y combinaciones válidas

Un catálogo de recursos es una lista estructurada de los recursos que expone la API, con sus URIs, sus verbos permitidos y sus códigos de respuesta esperados. El catálogo sirve como contrato de referencia para el equipo y como base para la documentación automática. La estructura del catálogo incluye el nombre del recurso, la URI base, los verbos permitidos y los códigos de respuesta esperados para cada verbo. La validación automática del catálogo comprueba que las combinaciones de verbo y código son coherentes. La coherencia del catálogo es la base de la previsibilidad de la API.

Catálogo de recursos:

Recurso	URI base	Verbos permitidos
Orden	/api/v1/ordenes	GET, POST
Orden	/api/v1/ordenes/{id}	GET, PUT, PATCH, DELETE
LineaOrden	/api/v1/ordenes/{id}/lineas	GET, POST

Recurso	URI base	Verbos permitidos
Material	/api/v1/materiales	GET, POST

- **Línea 1:** Recurso | URI base | Verbos permitidos → cabecera de la tabla.
- **Línea 2:** Orden | /api/v1/ordenes | GET, POST → colección de órdenes.
- **Línea 3:** Orden | /api/v1/ordenes/{id} | GET, PUT, PATCH, DELETE → orden concreta.
- **Línea 4:** LineaOrden | /api/v1/ordenes/{id}/lineas | GET, POST → colección de líneas.
- **Línea 5:** Material | /api/v1/materiales | GET, POST → colección de materiales.

La validación automática del catálogo comprueba tres cosas: que los verbos permitidos sean coherentes con la URI, que los códigos de respuesta sean coherentes con los verbos y que las URIs sigan las reglas de diseño. La comprobación se realiza con una clase de validación que recorre el catálogo y devuelve la lista de incoherencias. La validación automática evita que un endpoint con un verbo incorrecto o un código de respuesta inadecuado llegue a producción. La validación se ejecuta en los tests y en el pipeline de compilación.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica los conceptos de recursos, verbos y códigos de respuesta en cada endpoint. Los recursos se identifican con URIs que siguen las cinco reglas del punto 2.1. Los verbos se eligen según su semántica. Los códigos de respuesta se eligen según la relación sistemática entre verbo y resultado. El formato problem details se usa en todos los errores. El catálogo de recursos se documenta y se valida automáticamente. La aplicación conjunta produce una API predecible y coherente.

Aplicación al proyecto del curso:

- **Recursos identificados** → Orden, LineaOrden, Material, Cliente.
- **Verbos por recurso** → GET, POST, PUT, PATCH, DELETE.
- **Códigos por verbo** → 200, 201, 204, 400, 404, 409, 422, 500.
- **Formato de error** → problem details.
- **Catálogo documentado** → docs/catalogo-recursos.md.
- **Validación automática** → clase ValidadorCatalogoRecursos.
- **Línea 1:** Recursos identificados → lista de recursos del dominio.
- **Línea 2:** Verbos por recurso → combinaciones verbo-URI.
- **Línea 3:** Códigos por verbo → relación sistemática verbo-código.
- **Línea 4:** Formato de error → problem details.
- **Línea 5:** Catálogo documentado → documento de catálogo.
- **Línea 6:** Validación automática → clase de validación.

La aplicación de estos conceptos prepara el proyecto para el punto 2.3, donde se estudiará la validación de peticiones con FluentValidation. La validación de peticiones se apoya en los recursos y en los códigos de respuesta definidos en el punto 2.2. El punto 2.3 usará los recursos del catálogo para validar las peticiones de creación y de modificación. El punto 2.2 establece la base que se aplicará en los dos puntos siguientes del módulo 2.

Resumen rápido de la teoría

- Un recurso es una entidad identificable que agrupa datos y operaciones desde la perspectiva del cliente.

- Las representaciones transmiten los recursos y se negocian con la cabecera Accept.
- Los verbos HTTP tienen tres propiedades: seguridad, idempotencia y cacheabilidad.
- La combinación de verbo y URI define la operación con semántica clara.
- La relación entre verbo y código de respuesta es sistemática y predecible.
- Los códigos 4xx indican errores del cliente y los 5xx errores del servidor.
- El formato problem details estructura las respuestas de error de forma uniforme.
- El catálogo de recursos documenta y valida las combinaciones de verbo y código.

Del concepto a ASP.NET Core 8: DELETE, estado e idempotencia

DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.2-recursos-verbos-codigos`. Busca la instrucción `MapDelete` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Punto 2.3 — Validación de peticiones

Módulo 2: Desarrollo de APIs REST con ASP.NET Core Objetivos de aprendizaje:

- Comprender por qué la validación de peticiones es obligatoria en una API REST.
- Identificar los tres niveles de validación: sintáctica, semántica y de negocio.
- Comprender el papel de FluentValidation en ASP.NET Core.
- Comprender la relación entre validación fallida, código 400 y problem details.
- Añadir al proyecto AceriaApp una clase de validación de peticiones con FluentValidation y un test unitario que verifique la validación.

Bloque 1 — La validación como contrato de entrada

La validación de peticiones es el proceso de comprobar que los datos recibidos cumplen las reglas definidas por el servicio. Sin validación, el servicio procesa datos incorrectos y produce resultados incorrectos o falla de forma imprevisible. La validación es un contrato de entrada: define qué datos acepta el servicio y qué datos rechaza. El contrato se documenta y se verifica en cada petición. La validación no es una tarea opcional ni una capa de defensa secundaria: es la primera línea de defensa de la API. Un servicio sin validación es un servicio que delega la responsabilidad de los datos al cliente.



Figura M02.2.3.1 — Recorrido de una petición por los tres niveles de validación.

Flujo de la validación:

- **Cliente** → Petición → Validación sintáctica → Validación semántica → Validación de negocio → Procesamiento → Respuesta
- **Línea 1:** Cliente → Petición → el cliente envía la petición al servidor.
- **Línea 2:** → Validación sintáctica → primera comprobación, formato correcto.
- **Línea 3:** → Validación semántica → segunda comprobación, significado correcto.
- **Línea 4:** → Validación de negocio → tercera comprobación, reglas del dominio.
- **Línea 5:** → Procesamiento → se procesa la petición si pasa las tres validaciones.
- **Línea 6:** → Respuesta → se devuelve el resultado al cliente.

La validación se organiza en tres niveles que se aplican de forma secuencial. El primer nivel es la validación sintáctica, que comprueba que el formato de la petición es correcto. El segundo nivel es la validación semántica, que comprueba que los valores tienen sentido en el contexto del recurso. El tercer nivel es la validación de negocio, que comprueba que los datos cumplen las reglas del dominio. Los tres niveles se aplican en orden: si falla el primero, no se aplican los siguientes. La aplicación secuencial evita procesar datos que no pueden ser válidos en ningún caso.

La ausencia de validación produce tres consecuencias medibles. La primera es el incremento de errores en producción, porque los datos incorrectos llegan al procesamiento. La segunda es la dificultad de diagnóstico, porque el error se manifiesta lejos de la causa. La tercera es la pérdida de confianza del consumidor, porque la API no garantiza un contrato estable. Las tres consecuencias se corrigen con la adopción sistemática de la validación en tres niveles. La validación no elimina todos los errores, pero garantiza que los datos procesados cumplen el contrato de entrada.

Bloque 2 — Validación sintáctica

La validación sintáctica comprueba que la petición tiene el formato correcto. Se aplica antes de cualquier otra validación. En una API REST, la validación sintáctica incluye la comprobación del tipo de contenido, la comprobación del formato JSON, la comprobación de los tipos de datos de los campos y la comprobación de la presencia de los campos obligatorios. Un JSON mal formado produce un código 400 con un mensaje que indica el error de sintaxis. Un campo con un tipo incorrecto produce un código 400 con un mensaje que indica el campo y el tipo esperado. La validación sintáctica la realiza el framework en la mayoría de los casos.

```
public record CrearOrdenRequest(
    string Cliente,
    string Material,
```

```
    int Cantidad);
```

- **Línea 1:** public record CrearOrdenRequest(→ declara el contrato de entrada.
- **Línea 2:** string Cliente, → campo obligatorio de tipo string.
- **Línea 3:** string Material, → campo obligatorio de tipo string.
- **Línea 4:** int Cantidad); → campo obligatorio de tipo entero.

La validación sintáctica se apoya en el sistema de tipos del lenguaje. Un record con tipos no anulables obliga al framework a rechazar peticiones que no incluyan los campos. Un record con tipos anulables permite que los campos sean opcionales. La elección entre tipos anulables y no anulables es una decisión de diseño que se documenta en el contrato. La validación sintáctica también se apoya en los atributos de enlace de parámetros, que indican de dónde se obtiene cada valor. La combinación de tipos y atributos define la primera línea de defensa de la API.

Del concepto a ASP.NET Core 8: Niveles de validación

ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Aplicación en el proyecto real. Este comportamiento está materializado en `Validacion/Validadores.cs` del estado `2.3-validacion-peticiones`. Busca la instrucción `RuleFor` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 3 — Validación semántica

La validación semántica comprueba que los valores de la petición tienen sentido en el contexto del recurso. Se aplica después de la validación sintáctica. En una API REST, la validación semántica incluye la comprobación de rangos, la comprobación de formatos específicos, la comprobación de longitudes máximas y mínimas y la comprobación de coherencia entre campos. Un campo Cantidad con valor negativo falla la validación semántica. Un campo Email con formato incorrecto falla la validación semántica. Un campo FechaFin anterior a FechaInicio falla la validación semántica. La validación semántica se implementa con reglas específicas para cada campo.

```
public class CrearOrdenRequestValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenRequestValidator()
    {
        RuleFor(x => x.Cliente)
            .NotEmpty().WithMessage("El cliente es obligatorio.")
            .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres.");

        RuleFor(x => x.Material)
            .NotEmpty().WithMessage("El material es obligatorio.")
            .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres.");

        RuleFor(x => x.Cantidad)
            .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.")
            .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000.");
    }
```

```
}
```

- **Línea 1:** public class CrearOrdenRequestValidator : AbstractValidator → declara el validador.
- **Línea 2:** { → abre el cuerpo de la clase.
- **Línea 3:** public CrearOrdenRequestValidator() → constructor del validador.
- **Línea 4:** { → abre el cuerpo del constructor.
- **Línea 5:** RuleFor(x => x.Cliente) → comienza la regla para el campo Cliente.
- **Línea 6:** .NotEmpty().WithMessage("El cliente es obligatorio.") → no puede estar vacío.
- **Línea 7:** .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres."); → longitud máxima.
- **Línea 8:** RuleFor(x => x.Material) → comienza la regla para el campo Material.
- **Línea 9:** .NotEmpty().WithMessage("El material es obligatorio.") → no puede estar vacío.
- **Línea 10:** .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres."); → longitud máxima.
- **Línea 11:** RuleFor(x => x.Cantidad) → comienza la regla para el campo Cantidad.
- **Línea 12:** .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.") → mayor que cero.
- **Línea 13:** .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000."); → límite superior.
- **Línea 14:** } → cierra el constructor.
- **Línea 15:** } → cierra la clase.

La validación semántica tiene dos consecuencias prácticas. La primera es que cada campo tiene reglas explícitas y documentadas. La segunda es que los mensajes de error son específicos y útiles para el cliente. La combinación de ambas mejora la experiencia del consumidor y reduce el tiempo de integración. La validación semántica no comprueba reglas de negocio: comprueba que los valores son coherentes en el contexto del recurso. Las reglas de negocio se aplican en el siguiente nivel.

Bloque 4 — Validación de negocio

La validación de negocio comprueba que los datos cumplen las reglas del dominio. Se aplica después de la validación semántica. En una API REST, la validación de negocio incluye la comprobación de unicidad, la comprobación de existencia de referencias y la comprobación de reglas específicas del dominio. Un Cliente que no existe falla la validación de negocio. Un Material que está bloqueado falla la validación de negocio. Una Orden con más líneas de las permitidas falla la validación de negocio. La validación de negocio se implementa en la capa de aplicación o en el dominio, no en el validador de la petición.

```
public class OrdenServicio
{
    private readonly AceriaDbContext _db;

    public OrdenServicio(AceriaDbContext db)
    {
        _db = db;
    }

    public async Task<bool> ClienteExisteAsync(string cliente)
    {
        return await _db.Clientes.AnyAsync(c => c.Nombre == cliente);
    }

    public async Task<bool> MaterialDisponibleAsync(string material, int cantidad)
    {
```

```
        var stock = await _db.Materiales
            .Where(m => m.Nombre == material)
            .Select(m => m.Stock)
            .FirstOrDefaultAsync();
        return stock >= cantidad;
    }
}
```

- **Línea 1:** public class OrdenServicio → declara el servicio de aplicación.
- **Línea 2:** { → abre el cuerpo de la clase.
- **Línea 3:** private readonly AceriaDbContext _db; → campo para el contexto de base de datos.
- **Línea 4:** public OrdenServicio(AceriaDbContext db) → constructor.
- **Línea 5:** { → abre el cuerpo del constructor.
- **Línea 6:** _db = db; → asigna el contexto al campo.
- **Línea 7:** } → cierra el constructor.
- **Línea 8:** public async Task ClienteExisteAsync(string cliente) → método de comprobación de cliente.
- **Línea 9:** { → abre el cuerpo del método.
- **Línea 10:** return await _db.Clientes.AnyAsync(c => c.Nombre == cliente); → comprueba existencia.
- **Línea 11:** } → cierra el método.
- **Línea 12:** public async Task MaterialDisponibleAsync(string material, int cantidad) → método de comprobación de stock.
- **Línea 13:** { → abre el cuerpo del método.
- **Línea 14:** var stock = await _db.Materiales → consulta el stock.
- **Línea 15:** .Where(m => m.Nombre == material) → filtra por nombre.
- **Línea 16:** .Select(m => m.Stock) → proyecta el stock.
- **Línea 17:** .FirstOrDefaultAsync(); → obtiene el primer resultado.
- **Línea 18:** return stock >= cantidad; → comprueba disponibilidad.
- **Línea 19:** } → cierra el método.
- **Línea 20:** } → cierra la clase.

La validación de negocio tiene dos consecuencias prácticas. La primera es que se separa de la validación sintáctica y semántica. La segunda es que requiere acceso a datos externos al propio validador. La separación permite que la validación de negocio se implemente en la capa de aplicación o en el dominio, donde tiene acceso a los repositorios y a los servicios necesarios. La validación de negocio no se realiza en el validador de la petición porque el validador no tiene acceso a la base de datos. La separación por niveles es la clave para mantener la validación como un contrato claro y mantenible.

Del concepto a ASP.NET Core 8: Validador semántico

CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Aplicación en el proyecto real. Este comportamiento está materializado en `Validacion/Validadores.cs` del estado `2.3-validacion-peticiones`. Busca la instrucción `CrearOrdenValidator` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Validacion/Validadores.cs`

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 5 — FluentValidation en ASP.NET Core

FluentValidation es una biblioteca para definir reglas de validación de forma fluida y testeable. Se integra con ASP.NET Core mediante la clase AbstractValidator y el método RuleFor. Cada regla se compone de una expresión lambda que selecciona el campo, un método de validación y un mensaje de error. La biblioteca ofrece más de treinta métodos de validación entre los que se incluyen NotEmpty, MaximumLength, GreaterThan, EmailAddress, RegularExpression, Must y Custom. La combinación de métodos permite expresar reglas complejas de forma legible.

```
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenRequestValidator>();
```

- **Línea 1:** builder.Services.AddValidatorsFromAssemblyContaining(); → registra todos los validadores del ensamblado.

La integración con ASP.NET Core se realiza en el contenedor de dependencias. El método AddValidatorsFromAssemblyContaining registra todos los validadores del ensamblado que contiene el tipo especificado. Los validadores se inyectan en los endpoints o en los manejadores de MediatR. La validación se ejecuta antes de la lógica de negocio y devuelve una lista de errores si falla. La lista se transforma en una respuesta problem details con código 400. La integración con el contenedor y con el formato de error es la que permite que la validación sea transparente para la lógica de negocio.

Bloque 6 — Integración de la validación con Minimal API

En Minimal API, la validación se integra con un filtro de endpoint o con un manejador de excepciones. El filtro se ejecuta antes del manejador del endpoint y devuelve una respuesta problem details si la validación falla. El manejador se ejecuta solo si la validación pasa. La integración con Minimal API requiere registrar el filtro o el manejador en el contenedor y aplicarlo a los endpoints que lo necesitan. La aplicación selectiva del filtro permite que solo los endpoints con validación lo usen.



Figura M02.2.3.6 — Ejecución del filtro antes del manejador Minimal API.

```
public class ValidationFilter<T> : IEndpointFilter where T : class
{
    private readonly IValidator<T> _validator;

    public ValidationFilter(IValidator<T> validator)
    {
        _validator = validator;
    }

    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request = context.Arguments.OfType<T>().FirstOrDefault();
        if (request is null)
        {
            return Results.BadRequest();
        }

        var result = await _validator.ValidateAsync(request);
        if (!result.IsValid)
        {
            var errores = result.Errors
                .GroupBy(e => e.PropertyName)
                .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray());

            return Results.ValidationProblem(errores);
        }

        return await next(context);
    }
}
```

- **Línea 1:** public class ValidationFilter : IEndpointFilter where T : class → declara el filtro genérico.
- **Línea 2:** { → abre el cuerpo de la clase.
- **Línea 3:** private readonly IValidator _validator; → campo para el validador.
- **Línea 4:** public ValidationFilter(IValidator validator) → constructor.
- **Línea 5:** { → abre el cuerpo del constructor.
- **Línea 6:** _validator = validator; → asigna el validador.
- **Línea 7:** } → cierra el constructor.
- **Línea 8:** public async ValueTask InvokeAsync(→ método de invocación del filtro.
- **Línea 9:** EndpointFilterInvocationContext context, → contexto de la petición.

- **Línea 10:** EndpointFilterDelegate next) → siguiente paso en la cadena.
- **Línea 11:** { → abre el cuerpo del método.
- **Línea 12:** var request = context.Arguments.OfType().FirstOrDefault(); → obtiene la petición.
- **Línea 13:** if (request is null) → comprueba si la petición es nula.
- **Línea 14:** { → abre el bloque de error.
- **Línea 15:** return Results.BadRequest(); → devuelve 400.
- **Línea 16:** } → cierra el bloque de error.
- **Línea 17:** var result = await _validator.ValidateAsync(request); → ejecuta la validación.
- **Línea 18:** if (!result.IsValid) → comprueba si hay errores.
- **Línea 19:** { → abre el bloque de error.
- **Línea 20:** var errores = result.Errors → comienza la agrupación de errores.
- **Línea 21:** .GroupBy(e => e.PropertyName) → agrupa por nombre de propiedad.
- **Línea 22:** .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray()); → construye el diccionario.
- **Línea 23:** return Results.ValidationProblem(errores); → devuelve 400 con problem details.
- **Línea 24:** } → cierra el bloque de error.
- **Línea 25:** return await next(context); → continúa la cadena.
- **Línea 26:** } → cierra el método.
- **Línea 27:** } → cierra la clase.

El filtro genérico se aplica a los endpoints que lo necesitan. La aplicación se realiza con la extensión AddEndpointFilter. El filtro obtiene el validador del contenedor de dependencias y ejecuta la validación. Si la validación falla, devuelve una respuesta problem details con código 400 y los errores agrupados por propiedad. Si la validación pasa, continúa la cadena y ejecuta el manejador del endpoint. La integración con Minimal API es transparente para la lógica de negocio.

Del concepto a ASP.NET Core 8: Filtro de endpoint

ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Aplicación en el proyecto real. Este comportamiento está materializado en `Validacion/ValidationFilter.cs` del estado `2.3-validacion-peticiones`. Busca la instrucción `InvokeAsync` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 7 — Mensajes de error y localización

Los mensajes de error de la validación deben ser claros y útiles para el cliente. Un mensaje como "Valor inválido" no aporta información. Un mensaje como "La cantidad debe ser mayor que cero" aporta información concreta. Los mensajes se definen en el validador con el método WithMessage. La localización de los mensajes permite que la API devuelva mensajes en el idioma del cliente. La localización se implementa con recursos o con un proveedor de mensajes. La adopción de mensajes específicos y localizados mejora la experiencia del consumidor y reduce el tiempo de integración.

```
RuleFor(x => x.Cantidad)
    .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.")
    .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000.");
```

- **Línea 1:** RuleFor(x => x.Cantidad) → comienza la regla para el campo Cantidad.
- **Línea 2:** .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.") → regla y mensaje.
- **Línea 3:** .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000."); → regla y mensaje.

La adopción de mensajes específicos tiene dos consecuencias prácticas. La primera es que el cliente sabe exactamente qué ha fallado. La segunda es que el soporte puede reproducir el error con el mensaje. La localización se aplica de forma opcional según los requisitos del proyecto. El curso usa mensajes en español sin localización porque el alcance no lo requiere. La adopción de mensajes específicos es independiente de la localización y se aplica en todos los casos.

Bloque 8 — Validación y códigos de respuesta

La validación fallida produce un código de respuesta específico. El código 400 Bad Request se usa cuando la petición no cumple el contrato de entrada. El código 422 Unprocessable Entity se usa cuando la petición es sintácticamente correcta pero semánticamente inválida. La elección entre 400 y 422 depende del nivel de validación que haya fallado. Si falla la validación sintáctica o semántica, se devuelve 400. Si falla la validación de negocio, se devuelve 422. La distinción permite al cliente distinguir entre un error de datos y un error de negocio.

Códigos de respuesta según el nivel de validación:

- **Sintáctica fallida** → 400 Bad Request.
- **Semántica fallida** → 400 Bad Request.
- **Negocio fallida** → 422 Unprocessable Entity.
- **Línea 1:** Sintáctica fallida → error de formato, código 400.
- **Línea 2:** Semántica fallida → error de valores, código 400.
- **Línea 3:** Negocio fallida → error de reglas, código 422.

La distinción entre 400 y 422 tiene dos consecuencias prácticas. La primera es que el cliente puede reaccionar de forma distinta a cada código. La segunda es que las herramientas de monitorización pueden separar los errores de datos de los errores de negocio. La separación permite generar alertas específicas para cada tipo de error. El curso usa 400 para errores sintácticos y semánticos y 422 para errores de negocio.

Del concepto a ASP.NET Core 8: Registrar no es ejecutar

AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.3-validacion-peticiones`. Busca la instrucción `AddEndpointFilter` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Validacion/ValidationFilter.cs`

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 9 — Validación y problem details

La respuesta de validación fallida usa el formato problem details con el campo errors que contiene el diccionario de errores por propiedad. El campo errors no forma parte del estándar original de la RFC 7807, pero se ha convertido en una extensión ampliamente adoptada. La estructura del campo errors es un diccionario donde la clave es el nombre de la propiedad y el valor es una lista de mensajes de error. La estructura permite al cliente procesar los errores de forma uniforme.

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Cliente": ["El cliente es obligatorio."],
    "Cantidad": ["La cantidad debe ser mayor que cero."]
  }
}
```

- **Línea 1:** { → abre el objeto de error.
- **Línea 2:** "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1", → tipo del error.
- **Línea 3:** "title": "One or more validation errors occurred.", → resumen.
- **Línea 4:** "status": 400, → código HTTP.
- **Línea 5:** "errors": { → abre el diccionario de errores.
- **Línea 6:** "Cliente": ["El cliente es obligatorio."], → error del campo Cliente.
- **Línea 7:** "Cantidad": ["La cantidad debe ser mayor que cero."] → error del campo Cantidad.
- **Línea 8:** } → cierra el diccionario de errores.
- **Línea 9:** } → cierra el objeto de error.

La estructura del campo errors tiene dos consecuencias prácticas. La primera es que el cliente puede mostrar los errores junto a los campos correspondientes. La segunda es que el cliente puede procesar los errores de forma uniforme sin analizar el mensaje. La adopción del formato problem details con el campo errors es la práctica más común en las APIs REST modernas. El curso usa este formato en todos los endpoints que devuelven errores de validación.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica la validación de peticiones en tres niveles. La validación sintáctica se aplica con tipos no anulables en los record de petición. La validación semántica se aplica con FluentValidation. La validación de negocio se aplica en la capa de aplicación con acceso a la base de datos. La integración con Minimal API se realiza con un filtro genérico. La respuesta de validación fallida usa problem details con código 400. La respuesta de validación de negocio fallida usa problem details con código 422.

Aplicación al proyecto del curso:

- **Sintáctica** → records con tipos no anulables.
- **Semántica** → FluentValidation.
- **Negocio** → capa de aplicación.
- **Integración** → filtro genérico.
- **Respuesta** → problem details 400 o 422.
- **Línea 1:** Sintáctica → tipos no anulables en los records.
- **Línea 2:** Semántica → reglas con FluentValidation.
- **Línea 3:** Negocio → comprobaciones en la capa de aplicación.
- **Línea 4:** Integración → filtro genérico en Minimal API.
- **Línea 5:** Respuesta → problem details con 400 o 422.

La aplicación de la validación prepara el proyecto para el punto 2.4, donde se estudiará la documentación OpenAPI de los endpoints validados. La documentación OpenAPI incluye los esquemas de petición y de respuesta, y los códigos de error. El punto 2.4 usará los validadores definidos en el punto 2.3 para generar la documentación automática. El punto 2.3 establece la base que se aplicará en el punto 2.4.

Resumen rápido de la teoría

- La validación es el contrato de entrada de la API y se aplica en tres niveles: sintáctica, semántica y de negocio.
- La validación sintáctica comprueba el formato y los tipos.
- La validación semántica comprueba los valores y las longitudes.
- La validación de negocio comprueba las reglas del dominio con acceso a datos.
- FluentValidation permite definir reglas de validación de forma fluida y testeable.
- La integración con Minimal API se realiza con un filtro genérico.
- La validación fallida produce problem details con código 400 o 422 según el nivel.
- El campo errors de problem details contiene el diccionario de errores por propiedad.

Del concepto a ASP.NET Core 8: Problem Details y conflictos

Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.3-validacion-peticiones`. Busca la instrucción `Results.Problem` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Punto 2.4 — Documentación mediante OpenAPI (Swagger)

Módulo 2: Desarrollo de APIs REST con ASP.NET Core Objetivos de aprendizaje:

- Comprender el estándar OpenAPI y su relación con Swagger.
- Comprender la generación automática de documentación a partir del código.
- Comprender la estructura de un documento OpenAPI y sus secciones principales.
- Comprender las anotaciones de metadatos en Minimal API.
- Añadir al proyecto AceriaApp la documentación OpenAPI con metadatos, ejemplos y versionado, y un test unitario que verifique la documentación. Cerrar el módulo 2 con el resumen del estado del proyecto.

Bloque 1 — El estándar OpenAPI

OpenAPI es una especificación para describir APIs REST de forma legible por máquinas y por personas. La especificación define la estructura de un documento que describe los endpoints, los parámetros, los cuerpos de petición, las respuestas y los esquemas de datos. El documento se escribe en JSON o en YAML. La especificación se originó en el proyecto Swagger y fue donada a la OpenAPI Initiative en 2015. La versión actual es la 3.1, aunque la 3.0 sigue siendo la más adoptada. OpenAPI no es un framework ni una biblioteca: es un formato de descripción. Las herramientas que lo consumen pueden generar documentación, clientes, servidores de prueba y tests automáticos.

```
Estructura de un documento OpenAPI:

openapi: 3.0.3
info:
  title: AceriaApp.Ordenes.Api
  version: v1
paths:
  /api/v1/ordenes:
    get:
      summary: Lista las órdenes
      responses:
        '200':
          description: Lista de órdenes
```

- **Línea 1:** openapi: 3.0.3 → versión de la especificación.
- **Línea 2:** info: → sección de metadatos de la API.

- **Línea 3:** title: AceriaApp.Ordenes.Api → título de la API.
- **Línea 4:** version: v1 → versión del contrato.
- **Línea 5:** paths: → sección de endpoints.
- **Línea 6:** /api/v1/ordenes: → ruta del endpoint.
- **Línea 7:** get: → método HTTP.
- **Línea 8:** summary: Lista las órdenes → resumen del endpoint.
- **Línea 9:** responses: → sección de respuestas.
- **Línea 10:** '200': → código de respuesta.
- **Línea 11:** description: Lista de órdenes → descripción de la respuesta.

Un documento OpenAPI describe la API de forma completa y estructurada. La estructura permite que las herramientas generen documentación interactiva, clientes en distintos lenguajes y pruebas automáticas. La generación automática evita la documentación obsoleta porque el documento se deriva del código. La derivación del código no es automática por defecto: requiere que el código incluya los metadatos necesarios. Los metadatos se añaden con anotaciones, con atributos o con configuración explícita. El curso usa las anotaciones de Minimal API y la configuración de SwaggerGen.

Bloque 2 — Swagger y Swagger UI

Swagger es el nombre original del proyecto que dio lugar a OpenAPI. En la práctica, el término Swagger se usa para referirse a tres cosas: el formato de descripción, las herramientas de generación y la interfaz de usuario. La interfaz de usuario se llama Swagger UI y permite explorar la API de forma interactiva. Swagger UI lee el documento OpenAPI generado por el servidor y construye una página web que muestra los endpoints, los parámetros y las respuestas. La página incluye un botón para probar cada endpoint directamente desde el navegador. La combinación de OpenAPI y Swagger UI proporciona documentación viva y ejecutable.

Cómo surge un documento OpenAPI real



La flecha indica una relación de diseño o una secuencia de trabajo.

Figura M02.2.4.2 — Del endpoint de ASP.NET Core al documento OpenAPI.

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

- **Línea 1:** builder.Services.AddEndpointsApiExplorer(); → habilita la exploración de endpoints.

- **Línea 2:** builder.Services.AddSwaggerGen(); → registra el generador de Swagger.
- **Línea 3:** var app = builder.Build(); → construye la aplicación.
- **Línea 4:** if (app.Environment.IsDevelopment()) → solo en desarrollo.
- **Línea 5:** { → abre el bloque.
- **Línea 6:** app.UseSwagger(); → habilita el endpoint del documento OpenAPI.
- **Línea 7:** app.UseSwaggerUI(); → habilita la interfaz gráfica.
- **Línea 8:** } → cierra el bloque.

La configuración de Swagger en ASP.NET Core se realiza en dos fases. La primera fase registra los servicios con AddEndpointsApiExplorer y AddSwaggerGen. La segunda fase habilita los middleware con UseSwagger y UseSwaggerUI. La primera fase se ejecuta antes de builder.Build(). La segunda fase se ejecuta después. La separación entre las dos fases es una consecuencia del modelo de arranque de ASP.NET Core. El curso configura Swagger solo en el entorno de desarrollo para no exponer la documentación en producción.

Del concepto a ASP.NET Core 8: Generación real

OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Aplicación en el proyecto real. Este comportamiento está materializado en `Documentacion/DocumentacionExtensions.cs` del estado `2.4-documentacion-openapi`. Busca la instrucción `AddSwaggerGen` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 3 — Estructura de un documento OpenAPI

Un documento OpenAPI se organiza en secciones. La sección info contiene los metadatos de la API: título, versión, descripción y contacto. La sección servers contiene las URL base de la API. La sección paths contiene los endpoints con sus métodos y parámetros. La sección components contiene los esquemas de datos reutilizables. La sección security contiene los esquemas de autenticación. La sección tags contiene las etiquetas que agrupan los endpoints. Cada sección tiene una estructura definida por la especificación. La estructura es la misma independientemente del lenguaje o del framework que genere el documento.

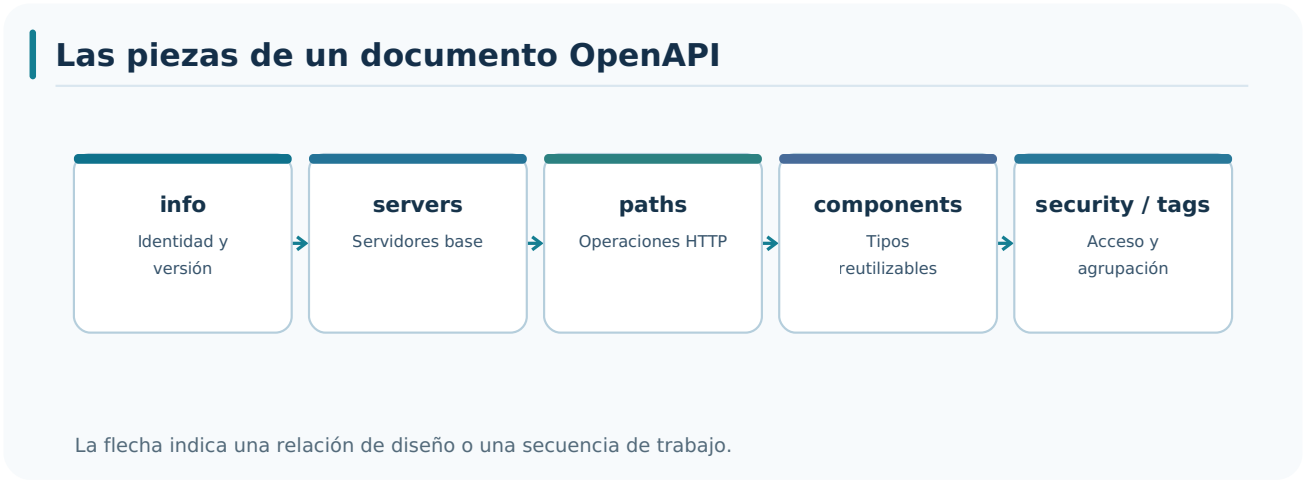


Figura M02.2.4.3 — Secciones que componen el contrato OpenAPI.

```
openapi: 3.0.3
info:
  title: AceriaApp.Monolito.Api
  version: v1
  description: API del monolito de referencia.
servers:
  - url: https://localhost:7000
paths:
  /api/v1/ordenes:
    get:
      tags:
        - Ordenes
      summary: Lista las órdenes
      responses:
        '200':
          description: Lista de órdenes
components:
  schemas:
    OrdenDto:
      type: object
      properties:
        id:
          type: string
          format: uuid
        cliente:
          type: string
```

- **Línea 1:** openapi: 3.0.3 → versión de la especificación.
- **Línea 2:** info: → metadatos de la API.
- **Línea 3:** title: AceriaApp.Monolito.Api → título.
- **Línea 4:** version: v1 → versión del contrato.
- **Línea 5:** description: API del monolito de referencia. → descripción.
- **Línea 6:** servers: → URL base de la API.
- **Línea 7:** - url: https://localhost:7000 → URL del servidor.
- **Línea 8:** paths: → endpoints.
- **Línea 9:** /api/v1/ordenes: → ruta del endpoint.
- **Línea 10:** get: → método HTTP.
- **Línea 11:** tags: → etiquetas del endpoint.
- **Línea 12:** - Ordenes → etiqueta del endpoint.
- **Línea 13:** summary: Lista las órdenes → resumen.
- **Línea 14:** responses: → respuestas.

- **Línea 15:** '200': → código de respuesta.
- **Línea 16:** description: Lista de órdenes → descripción de la respuesta.
- **Línea 17:** components: → esquemas reutilizables.
- **Línea 18:** schemas: → esquemas de datos.
- **Línea 19:** OrdenDto: → nombre del esquema.
- **Línea 20:** type: object → tipo del esquema.
- **Línea 21:** properties: → propiedades del esquema.
- **Línea 22:** id: → primera propiedad.
- **Línea 23:** type: string → tipo de la propiedad.
- **Línea 24:** format: uuid → formato de la propiedad.
- **Línea 25:** cliente: → segunda propiedad.
- **Línea 26:** type: string → tipo de la propiedad.

La estructura del documento OpenAPI se organiza de forma jerárquica. La sección paths contiene un objeto por ruta. Cada ruta contiene un objeto por método. Cada método contiene los parámetros, el cuerpo de petición y las respuestas. La sección components contiene los esquemas reutilizables que se referencian desde los endpoints con $ref. La reutilización de esquemas evita la duplicación de definiciones. La estructura jerárquica del documento refleja la estructura de la API y facilita la navegación.

Bloque 4 — Anotaciones de metadatos en Minimal API

Minimal API permite añadir metadatos a los endpoints con métodos de extensión. Los métodos más usados son WithName, WithSummary, WithDescription, WithTags, Produces y ProducesProblem. El método WithName asigna un nombre único al endpoint. El método WithSummary añade un resumen corto. El método WithDescription añade una descripción larga. El método WithTags agrupa el endpoint bajo una etiqueta. El método Produces declara el tipo de respuesta. El método ProducesProblem declara el tipo de respuesta de error.

```
app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes.AsNoTracking().ToListAsync())
    .WithName("ListarOrdenes")
    .WithSummary("Lista todas las órdenes")
    .WithDescription("Devuelve la lista completa de órdenes de producción.")
    .WithTags("Ordenes")
    .Produces<List<Orden>>(StatusCodes.Status200OK)
    .ProducesProblem(StatusCodes.Status500InternalServerError);
```

- **Línea 1:** app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) => → declara el endpoint.
- **Línea 2:** await db.Ordenes.AsNoTracking().ToListAsync()) → devuelve la lista.
- **Línea 3:** .WithName("ListarOrdenes") → asigna un nombre único.
- **Línea 4:** .WithSummary("Lista todas las órdenes") → añade un resumen.
- **Línea 5:** .WithDescription("Devuelve la lista completa de órdenes de producción.") → añade una descripción.
- **Línea 6:** .WithTags("Ordenes") → agrupa el endpoint bajo la etiqueta.
- **Línea 7:** .Produces<List>(StatusCodes.Status200OK) → declara el tipo de respuesta.
- **Línea 8:** .ProducesProblem(StatusCodes.Status500InternalServerError); → declara el tipo de error.

Las anotaciones tienen dos consecuencias prácticas. La primera es que el documento OpenAPI se genera con metadatos completos. La segunda es que Swagger UI muestra la información en la interfaz. La combinación de ambas mejora la experiencia del consumidor. Las anotaciones se aplican

de forma consistente en todos los endpoints. La consistencia es lo que permite que la documentación sea uniforme y predecible.

Del concepto a ASP.NET Core 8: Conexión en Program.cs

La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.4-documentacion-openapi`. Busca la instrucción `AddDocumentacion` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 5 — Esquemas de datos y DTOs

Los esquemas de datos se derivan de los tipos usados en los endpoints. Un record de respuesta produce un esquema con sus propiedades. Un record de petición produce un esquema con sus propiedades. Los esquemas se incluyen en la sección components del documento OpenAPI. Los endpoints referencian los esquemas con $ref. La referencia evita la duplicación de definiciones. La derivación de esquemas a partir de los tipos requiere que los tipos sean públicos y que sus propiedades sean accesibles. Los tipos anidados se incluyen como esquemas separados.

```
public record OrdenDto(
    Guid Id,
    string Cliente,
    string Material,
    int Cantidad,
    DateTime FechaCreacion);

app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes
        .Select(o => new OrdenDto(o.Id, o.Cliente, o.Material, o.Cantidad, o.FechaCreacion))
        .ToListAsync())
    .Produces<List<OrdenDto>>(StatusCodes.Status200OK);
```

- **Línea 1:** public record OrdenDto(→ declara el esquema de respuesta.
- **Línea 2:** Guid Id, → identificador de la orden.
- **Línea 3:** string Cliente, → nombre del cliente.
- **Línea 4:** string Material, → material solicitado.
- **Línea 5:** int Cantidad, → cantidad solicitada.
- **Línea 6:** DateTime FechaCreacion); → fecha de creación.
- **Línea 7:** app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) => → declara el endpoint.
- **Línea 8:** await db.Ordenes → consulta las órdenes.
- **Línea 9:** .Select(o => new OrdenDto(...)) → proyecta la entidad al DTO.
- **Línea 10:** .ToListAsync()) → materializa la lista.
- **Línea 11:** .Produces<List>(StatusCodes.Status200OK); → declara el tipo de respuesta.

La derivación de esquemas a partir de los tipos tiene dos consecuencias prácticas. La primera es que el contrato público está documentado de forma exacta. La segunda es que la evolución del contrato se refleja automáticamente en la documentación. La separación entre entidad y DTO del punto 2.1 se mantiene: los esquemas se derivan de los DTOs, no de las entidades. La separación garantiza que la documentación no expone la implementación interna del servicio.

Bloque 6 — Ejemplos en la documentación

Los ejemplos en la documentación muestran al consumidor cómo usar la API. Los ejemplos se añaden con anotaciones o con configuración explícita. Un ejemplo puede ser de petición o de respuesta. Un ejemplo de petición muestra el JSON que el cliente debe enviar. Un ejemplo de respuesta muestra el JSON que el servidor devuelve. Los ejemplos facilitan la integración y reducen el tiempo de prueba. La adopción de ejemplos es una buena práctica de documentación.

```
components:
  schemas:
    CrearOrdenRequest:
```

```
      type: object
      properties:
        cliente:
          type: string
          example: ACX-001
        material:
          type: string
          example: CHAPA-3MM
        cantidad:
          type: integer
          example: 10
```

- **Línea 1:** components: → sección de esquemas reutilizables.
- **Línea 2:** schemas: → esquemas de datos.
- **Línea 3:** CrearOrdenRequest: → esquema de petición.
- **Línea 4:** type: object → tipo del esquema.
- **Línea 5:** properties: → propiedades del esquema.
- **Línea 6:** cliente: → primera propiedad.
- **Línea 7:** type: string → tipo de la propiedad.
- **Línea 8:** example: ACX-001 → ejemplo del valor.
- **Línea 9:** material: → segunda propiedad.
- **Línea 10:** type: string → tipo de la propiedad.
- **Línea 11:** example: CHAPA-3MM → ejemplo del valor.
- **Línea 12:** cantidad: → tercera propiedad.
- **Línea 13:** type: integer → tipo de la propiedad.
- **Línea 14:** example: 10 → ejemplo del valor.

Los ejemplos tienen dos consecuencias prácticas. La primera es que el consumidor ve cómo se usa la API sin necesidad de leer toda la documentación. La segunda es que las herramientas pueden generar clientes con ejemplos precargados. La adopción de ejemplos es especialmente útil en APIs con contratos complejos. El curso usa ejemplos en los esquemas de petición y de respuesta.

Del concepto a ASP.NET Core 8: Metadatos y códigos

WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.4-documentacion-openapi`. Busca la instrucción `Produces` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 7 — Versionado de la documentación

La documentación se versiona junto con el contrato. Cada versión del contrato tiene su propio documento OpenAPI. Swagger UI permite seleccionar la versión que se quiere consultar. La selección se realiza con el método SwaggerDoc que registra cada versión. La versión de la documentación

coincide con la versión del contrato en la URL. La coincidencia permite que el consumidor consulte la documentación de la versión que usa. El versionado de la documentación es una consecuencia del versionado del contrato.

```
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "AceriaApp.Monolito.Api",
        Version = "v1",
        Description = "API del monolito de referencia."
    });
});

app.UseSwaggerUI(options =>
{
    options.SwaggerEndpoint("/swagger/v1/swagger.json", "AceriaApp.Monolito.Api v1");
});
```

- **Línea 1:** builder.Services.AddSwaggerGen(options => → registra el generador.
- **Línea 2:** { → abre el bloque.
- **Línea 3:** options.SwaggerDoc("v1", new OpenApiInfo → registra la versión v1.
- **Línea 4:** { → abre el bloque de metadatos.
- **Línea 5:** Title = "AceriaApp.Monolito.Api", → título de la API.
- **Línea 6:** Version = "v1", → versión del contrato.
- **Línea 7:** Description = "API del monolito de referencia." → descripción.
- **Línea 8:** }); → cierra el bloque.
- **Línea 9:** }); → cierra la configuración.
- **Línea 10:** app.UseSwaggerUI(options => → configura Swagger UI.
- **Línea 11:** { → abre el bloque.
- **Línea 12:** options.SwaggerEndpoint("/swagger/v1/swagger.json", "AceriaApp.Monolito.Api v1"); → registra el endpoint del documento.
- **Línea 13:** }); → cierra el bloque.

El versionado de la documentación tiene dos consecuencias prácticas. La primera es que el consumidor puede consultar la versión que usa. La segunda es que la documentación de versiones antiguas sigue disponible mientras el contrato antiguo siga activo. La combinación de ambas permite la evolución sin romper a los consumidores. El curso usa el versionado en la URL y en la documentación de forma coherente.

Bloque 8 — Documentación de errores y problem details

La documentación incluye los códigos de error y el formato problem details. El método ProducesProblem declara el tipo de error. El método ProducesValidationProblem declara el tipo de error de validación. La documentación muestra los códigos de error esperados para cada endpoint. La inclusión de los errores en la documentación permite al consumidor anticipar las respuestas. La anticipación reduce el tiempo de integración y mejora la experiencia del consumidor.

```
app.MapPost("/api/v1/ordenes", async (CrearOrdenRequest request, AceriaDbContext db) =>
{
    // lógica del endpoint
})
.WithName("CrearOrden")
.WithSummary("Crea una orden")
.Produces<OrdenDto>(StatusCodes.Status201Created)
```

```
.ProducesValidationProblem(StatusCodes.Status400BadRequest)
.ProducesProblem(StatusCodes.Status500InternalServerError);
```

- **Línea 1:** app.MapPost("/api/v1/ordenes", async (CrearOrdenRequest request, AceriaDbContext db) => → declara el endpoint POST.
- **Línea 2:** { → abre el cuerpo del lambda.
- **Línea 3:** // lógica del endpoint → comentario del cuerpo.
- **Línea 4:** }) → cierra el lambda.
- **Línea 5:** .WithName("CrearOrden") → asigna un nombre único.
- **Línea 6:** .WithSummary("Crea una orden") → añade un resumen.
- **Línea 7:** .Produces(StatusCodes.Status201Created) → declara la respuesta exitosa.
- **Línea 8:** .ProducesValidationProblem(StatusCodes.Status400BadRequest) → declara el error de validación.
- **Línea 9:** .ProducesProblem(StatusCodes.Status500InternalServerError); → declara el error interno.

La documentación de errores tiene dos consecuencias prácticas. La primera es que el consumidor ve los códigos de error esperados. La segunda es que las herramientas pueden generar clientes que manejen cada código de forma específica. La adopción de la documentación de errores es una buena práctica que se aplica de forma uniforme en todos los endpoints. El curso documenta los códigos 400, 404, 409, 422 y 500 según corresponda.

Del concepto a ASP.NET Core 8: Ejemplos y contratos

SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Aplicación en el proyecto real. Este comportamiento está materializado en `Documentacion/DocumentacionExtensions.cs` del estado `2.4-documentacion-openapi`. Busca la instrucción `EjemploCrearOrden` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Código de referencia probado: `src/AceriaApp.Api/Program.cs`

```
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
```

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.

Bloque 9 — Herramientas que consumen OpenAPI

El documento OpenAPI lo consumen varias herramientas. Swagger UI lo consume para generar la documentación interactiva. OpenAPI Generator lo consume para generar clientes en distintos lenguajes. Postman lo consume para importar la colección de endpoints. ReDoc lo consume para generar documentación de solo lectura. Las herramientas se integran con el documento sin modificar la API. La integración es posible porque el documento es un formato estándar. La estandarización es la que permite que la documentación se reutilice en múltiples herramientas.

```
Herramientas que consumen OpenAPI:
```

```
Swagger UI           → documentación interactiva.
OpenAPI Generator    → clientes en varios lenguajes.
Postman              → colección de endpoints.
ReDoc                → documentación de solo lectura.
```

- **Línea 1:** Swagger UI → documentación interactiva.
- **Línea 2:** OpenAPI Generator → clientes en varios lenguajes.
- **Línea 3:** Postman → colección de endpoints.
- **Línea 4:** ReDoc → documentación de solo lectura.

La integración con herramientas tiene dos consecuencias prácticas. La primera es que la documentación se reutiliza en distintos contextos. La segunda es que el consumidor elige la herramienta que mejor se adapta a su flujo de trabajo. La combinación de ambas reduce el coste de integración y mejora la experiencia del consumidor. El curso usa Swagger UI como herramienta principal y menciona las demás como opciones.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica la documentación OpenAPI en todos los endpoints. Los metadatos se añaden con las anotaciones de Minimal API. Los esquemas se derivan de los DTOs. Los ejemplos se añaden en los esquemas de petición. La versión del contrato coincide con la versión de la URL. Los códigos de error se documentan con ProducesProblem y ProducesValidationProblem. La documentación se publica con Swagger UI en el entorno de desarrollo. La documentación se cierra con un test que verifica la generación del documento.

```
Aplicación al proyecto del curso:

Metadatos        → WithName, WithSummary, WithDescription, WithTags.
Esquemas         → derivados de los DTOs.
Ejemplos         → añadidos en los esquemas de petición.
Versionado       → v1 en URL y en documentación.
Errores          → ProducesProblem y ProducesValidationProblem.
Publicación      → Swagger UI en desarrollo.
Verificación     → test de generación del documento.
```

- **Línea 1:** Metadatos → anotaciones de Minimal API.
- **Línea 2:** Esquemas → derivados de los DTOs.
- **Línea 3:** Ejemplos → añadidos en los esquemas de petición.
- **Línea 4:** Versionado → v1 en URL y en documentación.
- **Línea 5:** Errores → documentados con ProducesProblem y ProducesValidationProblem.
- **Línea 6:** Publicación → Swagger UI en desarrollo.
- **Línea 7:** Verificación → test de generación del documento.

La aplicación de la documentación cierra el módulo 2 y prepara el terreno para el módulo 3. El módulo 3 estudiará la arquitectura hexagonal, que reorganizará el código en capas con puertos y adaptadores. La documentación OpenAPI seguirá generándose a partir de los endpoints, independientemente de la organización interna. El punto 2.4 cierra el módulo 2 con el resumen del estado del proyecto.

Resumen rápido de la teoría

- OpenAPI es una especificación para describir APIs REST de forma legible por máquinas y personas.
- Swagger UI lee el documento OpenAPI y construye una interfaz interactiva.
- Un documento OpenAPI se organiza en secciones: info, servers, paths, components, security y tags.

- Minimal API permite añadir metadatos con WithName, WithSummary, WithDescription y WithTags.
- Los esquemas de datos se derivan de los DTOs y se referencian con $ref.
- Los ejemplos en los esquemas facilitan la integración del consumidor.
- El versionado de la documentación coincide con el versionado del contrato en la URL.
- Los errores se documentan con ProducesProblem y ProducesValidationProblem.

Del concepto a ASP.NET Core 8: Comprobar OpenAPI por HTTP

Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor.

Aplicación en el proyecto real. Este comportamiento está materializado en `Program.cs` del estado `2.4-documentacion-openapi`. Busca la instrucción `UseDocumentacion` y sigue su ejecución con un punto de interrupción. En las prácticas no se pedirá únicamente crear una clase: el alumno construirá el recurso, enviará la petición y observará el código HTTP y, cuando proceda, la escritura en SQL Server Express LocalDB.

Qué debe saber explicar el alumno. Qué parte del contrato REST se resuelve en ASP.NET Core, qué parte escribe el programador, qué objeto se inyecta, qué ocurre si la entrada falla y por qué la respuesta HTTP es la indicada. Esta comprobación enlaza la explicación teórica con las acciones posteriores.